

Heaps and Sweep Line

23. Merge k Sorted Lists

Merging k sorted lists is a cornerstone problem for senior engineers (L5/L6). At this level, the interviewer is looking for more than just a correct answer—they want to see you weigh the trade-offs between **Min-Heaps** and **Divide and Conquer**, handle memory management for large datasets, and write production-quality code.

1. Problem Explanation

The Goal: You are given an array of k linked lists, where each individual list is already sorted in ascending order. You need to merge all of them into a single, massive sorted linked list.

The Intuition: Imagine you have 5 decks of cards on a table. Each deck is already sorted from Ace to King. You want to create one big pile that is also sorted. * **The naive way:** Take all the cards, throw them in a pile, and then sort the whole pile. (Very slow). * **The senior way:** Look at the top card of every deck. Pick the smallest one, put it in the new pile, and then look at the *new* top card of that specific deck. Repeat until all decks are empty.

Example: Input: List 1: 1 -> 4 -> 5 List 2: 1 -> 3 -> 4 List 3: 2 -> 6

Output: 1 -> 1 -> 2 -> 3 -> 4 -> 4 -> 5 -> 6

2. Solution Explanation

Approach: The Min-Heap (Priority Queue)

The “Min-Heap” is the most robust way to solve this. It acts like a “sorting machine” that always keeps the smallest available item at the very top.

The Steps: 1. Create a Min-Heap. 2. Take the very first node (the **head**) of every list and put it into the heap. 3. While the heap is not empty: * Extract the smallest node from the heap (the top). * Add this node to your “Result” linked list. * If that node had a “next” node in its original list, put that next node into the heap. 4. Once the heap is empty, your result list is complete.

Detailed ASCII Walkthrough:

Lists: L1: [1] -> 4 L2: [2] -> 5 L3: [0] -> 9

[INITIAL STATE]

Heap: [(0, L3), (1, L1), (2, L2)] <-- Sorted by value

Result List: (Dummy Head)

```

[ STEP 1 ]
1. Pop smallest from Heap: (0, L3)
2. Result: Dummy -> 0
3. L3's next node is 9. Push (9, L3_next) to Heap.

Heap: [(1, L1), (2, L2), (9, L3_next)]
-----

[ STEP 2 ]
1. Pop smallest from Heap: (1, L1)
2. Result: Dummy -> 0 -> 1
3. L1's next node is 4. Push (4, L1_next) to Heap.

Heap: [(2, L2), (4, L1_next), (9, L3_next)]
-----

[ STEP 3 ]
1. Pop smallest from Heap: (2, L2)
2. Result: Dummy -> 0 -> 1 -> 2
3. L2's next node is 5. Push (5, L2_next) to Heap.

Heap: [(4, L1_next), (5, L2_next), (9, L3_next)]
-----

[ STEP 4 ]
1. Pop smallest: 4. Add to Result.
2. 4 has no 'next'. Heap size shrinks.
... repeat until empty ...

Final Result: 0 -> 1 -> 2 -> 4 -> 5 -> 9

```

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(N * \log k)$

Let N = Total number of nodes across ALL lists.

Let k = Total number of lists.

1. Each node is pushed into the heap once: $O(N)$ operations.
2. Each node is popped from the heap once: $O(N)$ operations.
3. Every heap operation (push/pop) takes $O(\log k)$ time because the heap never contains more than 'k' elements at a time.

TOTAL TIME: $O(N * \log k)$

[TC Visual Comparison]

Brute Force Sort: $O(N * \log N)$

Heap Approach: $O(N * \log k)$
(Since k is usually much smaller than N , the Heap is much faster)

SPACE COMPLEXITY: $O(k)$

1. We are creating a new result list, but we are reusing the existing nodes (just changing their pointers), so we ignore the output space.
2. The extra space used is the Heap itself.
3. The heap stores exactly one node from each of the ' k ' lists.

TOTAL SPACE: $O(k)$

[Space Visual]

Heap Storage: [Node_L1 | Node_L2 | ... | Node_Lk]
(Size remains constant at ' k ' nodes)

4. Solution Code

Python Snippet (Using heapq)

```
import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

    # We need this for the heap to compare nodes
    def __lt__(self, other):
        return self.val < other.val

def mergeKLists(lists):
    # Dummy node to act as the starting point of our result
    dummy = ListNode(0)
    current = dummy

    # Initialize the heap with the head of each list
    heap = []
    for l in lists:
        if l:
            heapq.heappush(heap, l)
```

```

# Process the heap until empty
while heap:
    # Get the node with the smallest value
    node = heapq.heappop(heap)

    # Link it to our resulting list
    current.next = node
    current = current.next

    # If the node has a successor, add it to the heap
    if node.next:
        heapq.heappush(heap, node.next)

return dummy.next

```

JavaScript Snippet (Using a custom Min-Heap logic)

```

// JavaScript doesn't have a built-in Priority Queue/Heap like Python.
// A senior engineer would likely implement a basic one or use
// the Divide and Conquer approach to avoid custom heap boilerplate.

function mergeKLists(lists) {
    if (lists.length === 0) return null;

    // Divide and Conquer Approach: Merge lists in pairs
    // This also achieves  $O(N * \log k)$  time and  $O(1)$  extra space (iterative)
    while (lists.length > 1) {
        let mergedLists = [];
        for (let i = 0; i < lists.length; i += 2) {
            let l1 = lists[i];
            let l2 = i + 1 < lists.length ? lists[i + 1] : null;
            mergedLists.push(mergeTwoLists(l1, l2));
        }
        lists = mergedLists;
    }

    return lists[0];
}

/**
 * Standard function to merge two sorted linked lists
 */
function mergeTwoLists(l1, l2) {
    let dummy = { val: 0, next: null };
    let current = dummy;

```

```

while (l1 && l2) {
    if (l1.val < l2.val) {
        current.next = l1;
        l1 = l1.next;
    } else {
        current.next = l2;
        l2 = l2.next;
    }
    current = current.next;
}
current.next = l1 || l2;
return dummy.next;
}

```

Note 1: Terminology Addendum

Min-Heap (Priority Queue): * **What it is:** A specialized tree-based data structure that satisfies the heap property: the root is always the minimum element. * **Why it helps:** It allows you to find and remove the smallest element in $O(\log k)$ time, which is much faster than re-sorting the whole list every time. * **Application:** Here, it keeps track of the “smallest available” head across all k lists efficiently.

Divide and Conquer: * **What it is:** Breaking a large problem into two smaller sub-problems, solving them, and combining the results. * **How it applies:** Instead of merging one list at a time (1+2, then result+3, then result+4), we merge 1+2 and 3+4 separately, then merge those results. This reduces the number of comparisons drastically.

Note 2: Real-World Interview Variations

1. Google: “External Sort” (Massive Data Variation) * **The Prompt:** You have 100GB of log files, but only 1GB of RAM. How do you sort all of it? * **The L5 Solve:** This is exactly the Merge K Sorted Lists problem. 1. Split the 100GB file into 100 small 1GB chunks. 2. Sort each 1GB chunk in memory and save it back to disk. 3. Now you have 100 sorted “lists” on disk. 4. Use a Min-Heap of size 100 to merge them, reading one line from each file at a time. This is called **Multi-way Merge Sort**.

2. Bloomberg: “Merge Real-time Stock Tickers” * **The Prompt:** You receive 50 different real-time streams of stock prices. Each stream is sorted by timestamp. Merge them into a single sorted dashboard. * **The L5 Solve:** Use a Min-Heap of size 50. As a new ticker arrives from a stream, it enters the

heap. This ensures the dashboard always displays the earliest ticker first across all sources.

3. Meta: “Merge K Sorted Iterators” * The Prompt: Instead of linked lists, you are given K custom Iterator objects with `next()` and `hasNext()` methods. * **The L5 Solve:** The logic remains the same. The “Next” node for the heap is simply `iterator.next()`. This tests your ability to abstract the problem beyond simple data structures into Object Oriented Programming.

347. Top K Frequent Elements

This is a classic problem that tests your ability to move from a basic solution to an optimized one by understanding data distribution. An L5/L6 Google engineer wouldn’t just give you a “Heap” solution; they would advocate for **Bucket Sort** because it hits a better time complexity for this specific constraint.

1. Problem Explanation

The Premise: You are given an array of integers `nums` and an integer `k`. You need to return the `k` most frequent elements.

The “Why” of the Challenge: If you just needed to find the most frequent element, you’d count them and pick the winner. But finding the “Top K” means you need to rank them. - A simple sort would take too long. - A Heap is better, but still involves “logarithmic” overhead. - **Bucket Sort** is the “Gold Standard” here because the maximum possible frequency of any element is capped by the size of the input array.

Example: `nums = [1, 1, 1, 2, 2, 3]`, `k = 2`. 1. Count the occurrences: 1 appears 3 times, 2 appears 2 times, 3 appears 1 time. 2. The Top 2 most frequent are 1 and 2.

2. Solution Explanation

We will use the **Bucket Sort** approach. Unlike traditional sorting that compares numbers, Bucket Sort uses the *frequency* as an index in an array.

The Strategy:

1. **Frequency Map:** Use a hash map to count how many times each number appears.
2. **The Buckets:** Create an array of lists (buckets). The index of the bucket represents the **frequency**.
 - If number `X` appears 3 times, put `X` into `buckets[3]`.

3. **Collect Results:** Iterate through the buckets from right to left (highest frequency to lowest) and collect elements until you have **k** items.

Detailed ASCII Walkthrough Input: `nums = [1, 1, 1, 2, 2, 100]`, `k = 2`

Step 1: Count Frequencies

```
Map:
{
  1: 3, (Value 1 appears 3 times)
  2: 2, (Value 2 appears 2 times)
  100: 1 (Value 100 appears 1 time)
}
```

Step 2: Initialize Buckets The array has a length of 6, so the max frequency is 6. We create buckets from index 0 to 6.

```
Index (Freq):  0    1    2    3    4    5    6
Bucket:       [ ] [ ] [ ] [ ] [ ] [ ] [ ]
```

Step 3: Fill Buckets * 100 has freq 1 -> Put in `buckets[1]` * 2 has freq 2 -> Put in `buckets[2]` * 1 has freq 3 -> Put in `buckets[3]`

```
Index (Freq):  0    1    2    3    4    5    6
Bucket:       [ ] [100] [2] [1] [ ] [ ] [ ]
```

Step 4: Extract Top K (k=2) We start from index 6 and move left: - Index 6, 5, 4: Empty. - Index 3: Found [1]. Result = [1]. (Need 1 more) - Index 2: Found [2]. Result = [1, 2]. (We have 2 elements, STOP)

Final Result: [1, 2]

3. Time and Space Complexity Analysis

Time Complexity: $O(N)$ Where **N** is the number of elements in `nums`.

TC Derivation:

1. Build Frequency Map:
Iterate through `nums` once.
Work: $O(N)$
2. Build Buckets:
Iterate through the Map (at most **N** unique elements).
Work: $O(N)$
3. Extract Top K:
Iterate through buckets (length **N**) and collect **K** elements.
Work: $O(N)$

Total Time = $O(N) + O(N) + O(N) = O(3N)$

Simplified: $O(N)$

Space Complexity: $O(N)$

SC Derivation:

1. Frequency Map:

Stores at most N unique elements.

Work: $O(N)$

2. Buckets Array:

An array of length N , containing N total elements.

Work: $O(N)$

Total Space: $O(N)$

4. Solution Code

Python Implementation (Bucket Sort)

```
def topKFrequent(nums, k):
    # 1. Count occurrences using a hash map
    count = {}
    for n in nums:
        count[n] = 1 + count.get(n, 0)

    # 2. Create 'buckets' where the index is the frequency
    # Example: freq[3] = [list of numbers that appeared 3 times]
    # We need len(nums) + 1 buckets to handle the max frequency case
    freq = [[] for i in range(len(nums) + 1)]

    for n, c in count.items():
        freq[c].append(n)

    # 3. Iterate from highest frequency to lowest
    res = []
    for i in range(len(freq) - 1, 0, -1):
        for n in freq[i]:
            res.append(n)
            if len(res) == k:
                return res
```

JavaScript Implementation (Bucket Sort)


```

var topKFrequent = function(nums, k) {
  const count = new Map();
  // 1. Build frequency map
  for (const n of nums) {
    count.set(n, (count.get(n) || 0) + 1);
  }

  // 2. Create buckets. Array of arrays.
  const buckets = Array.from({ length: nums.length + 1 }, () => []);

  for (const [n, c] of count.entries()) {
    buckets[c].push(n);
  }

  // 3. Collect from the end (highest freq)
  const res = [];
  for (let i = buckets.length - 1; i >= 0; i--) {
    if (buckets[i].length > 0) {
      for (const n of buckets[i]) {
        res.push(n);
        if (res.length === k) return res;
      }
    }
  }
};

```

Note 1: Terms and Techniques

- **Bucket Sort:** A non-comparative sorting algorithm. It works by distributing elements into “buckets” and is extremely efficient when the range of possible “keys” (in this case, frequencies) is small or bounded.
 - **Pigeonhole Principle:** This justifies why we only need $N+1$ buckets. Since you only have N total items, no single number can appear more than N times.
-

Note 2: Real-World Interview Variations

1. **Google: “Top K Trending Search Queries”** * **The Twist:** The data doesn’t fit in memory. You have billions of search logs. * **L5 Solution:** You can’t use Bucket Sort on a single machine. You’d use **MapReduce**. * **Map:** Count frequencies in chunks. * **Reduce:** Aggregate counts. * **Final step:** Use a **Min-Heap** of size K to track the global Top K across all reduced results. This keeps memory usage at $O(K)$.

2. Meta: “Most Frequent Words in a Live Stream” * The Twist: The data is a continuous stream. You need the Top K *right now*. * **L5 Solution:** Use a **Count-Min Sketch** (to approximate frequencies with very little memory) combined with a **Min-Heap** to track the current top elements. This is an “Approximate Algorithm” for high-scale streaming.

3. Bloomberg: “Top K Active Stock Tickers” * The Twist: You need to update the Top K every second as new trades come in. * **L5 Solution:** Use a combination of a **Hash Map** (Ticker -> Node) and a **Doubly Linked List** (frequency-based), similar to an **LRU Cache** but ordered by frequency. When a ticker’s frequency increases, you move its node forward in the list. This allows for $O(1)$ updates and $O(K)$ retrieval.

295. Find Median from Data Stream

A Google L5 or L6 engineer doesn’t just solve this problem; they architect it for scale. When an engineer sees the word “stream,” they immediately think about efficiency—specifically, how to avoid re-sorting data every time a new piece arrives.

Here is the masterclass walkthrough for LeetCode 295.

1. Problem Explanation

The Goal: You are receiving a continuous stream of numbers. At any point in time, you must be able to return the **median** of all elements seen so far.

What is a Median? * If the list length is **odd**, the median is the middle element of the sorted list. * If the list length is **even**, the median is the average of the two middle elements.

The Constraints (The “Why” it’s hard): A naive solution would be to store numbers in a list and sort them every time you need the median. * Sorting takes $O(N \log N)$ time. * If you do this for every new number, the total time for N numbers becomes $O(N^2 \log N)$. * **A senior engineer knows this will fail for a million data points.** We need a way to find the middle without keeping the whole list perfectly sorted.

2. Solution Explanation

The Core Insight: Two Heaps To find the median, we don’t need to know the order of *all* numbers. we only need to know the **middle** numbers.

Imagine splitting the data stream into two halves: 1. **The Small Half:** The smaller 50% of the numbers. 2. **The Large Half:** The larger 50% of the

numbers.

If we can quickly get the **largest** of the small half and the **smallest** of the large half, we have the median!

The Tools: * **Max-Heap (Small Half):** Stores the smaller half. We want easy access to the *largest* value in this group. * **Min-Heap (Large Half):** Stores the larger half. We want easy access to the *smallest* value in this group.

The Balancing Act: To keep the median in the center, we follow two rules:
1. **Size Rule:** The size difference between the two heaps must never be more than 1. 2. **Order Rule:** Every number in the Max-Heap must be less than or equal to every number in the Min-Heap.

ASCII Diagram Walkthrough Let's trace the stream: [5, 2, 10, 8]

Step 1: Add 5 Always try to add to the Max-Heap first.

Small Half (Max-Heap)		Large Half (Min-Heap)
[5]		[]

Median: 5 (Only one element)

Step 2: Add 2 2 is smaller than the top of the Max-Heap (5), so it stays in the Small Half.

Small Half (Max-Heap)		Large Half (Min-Heap)
[5, 2]		[]
^		

(Wait, size diff is 2! We must rebalance)

Rebalance: Move top of Max-Heap to Min-Heap

Small Half (Max-Heap)		Large Half (Min-Heap)
[2]		[5]

Median: $(2 + 5) / 2 = 3.5$

Step 3: Add 10 10 is larger than the top of the Small Half, so it goes to the Min-Heap.

Small Half (Max-Heap)		Large Half (Min-Heap)
[2]		[5, 10]

(Wait, size diff is 2 again! Move 5 back to Small Half)

Small Half (Max-Heap)		Large Half (Min-Heap)
[5, 2]		[10]

Median: 5 (The "extra" element in the larger-sized heap)

Step 4: Add 8 8 is larger than 5, so it goes to the Large Half (Min-Heap).

Small Half (Max-Heap)	Large Half (Min-Heap)
[5, 2]	[8, 10]

Median: $(5 + 8) / 2 = 6.5$

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(\log N)$ per add, $O(1)$ per median
N = Total numbers in the stream
1. addNum(): - Inserting into a heap takes $O(\log N)$. - Removing/Rebalancing from a heap takes $O(\log N)$. Total for one add: $O(\log N)$
2. findMedian(): - Looking at the top of a heap takes $O(1)$. Total for one lookup: $O(1)$
TOTAL FOR N ADDS: $O(N \log N)$ (Massive improvement over $O(N^2 \log N)$)
SPACE COMPLEXITY DERIVATION: $O(N)$
1. We store every number that arrives in the stream.
2. Half the numbers go to the Max-Heap ($N/2$).
3. Half the numbers go to the Min-Heap ($N/2$).
TOTAL SPACE: $O(N)$ to store the data points.

4. Solution Code

Note: JavaScript does not have a built-in Heap/Priority Queue class. In a real Google interview, an L5 would likely use a library or write a quick helper. Python has `heapq`.

Optimized Two-Heap Solution Python Snippet

```

import heapq

class MedianFinder:
    def __init__(self):
        # Small half: Max-Heap (we use negative values because heapq is a min-heap)
        self.small = []
        # Large half: Min-Heap
        self.large = []

    def addNum(self, num: int):
        # Step 1: Add to Small Half (Max-Heap)
        heapq.heappush(self.small, -num)

        # Step 2: Ensure the largest of small is <= smallest of large
        if self.small and self.large and (-self.small[0] > self.large[0]):
            val = -heapq.heappop(self.small)
            heapq.heappush(self.large, val)

        # Step 3: Handle Size Imbalance (Rebalance)
        if len(self.small) > len(self.large) + 1:
            val = -heapq.heappop(self.small)
            heapq.heappush(self.large, val)
        elif len(self.large) > len(self.small) + 1:
            val = heapq.heappop(self.large)
            heapq.heappush(self.small, -val)

    def findMedian(self) -> float:
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        elif len(self.large) > len(self.small):
            return float(self.large[0])
        else:
            # Even number of elements: average the tops
            return (-self.small[0] + self.large[0]) / 2.0

```

JavaScript Snippet (Conceptual - Using a MinPriorityQueue interface)

```

class MedianFinder {
    constructor() {
        // Assume MinPriorityQueue is available (common in modern interview environments)
        this.large = new MinPriorityQueue(); // Smallest of large half
        this.small = new MaxPriorityQueue(); // Largest of small half
    }

    /**
     * Adds a number to the stream and rebalances the heaps.

```

```

    */
    addNum(num) {
        // Step 1: Always start by pushing to the small half
        this.small.enqueue(num);

        // Step 2: Validate the Order Rule
        // If the max of small > min of large, swap them.
        if (this.small.size() > 0 && this.large.size() > 0 &&
            this.small.front().element > this.large.front().element) {
            this.large.enqueue(this.small.dequeue().element);
        }

        // Step 3: Validate the Size Rule
        if (this.small.size() > this.large.size() + 1) {
            this.large.enqueue(this.small.dequeue().element);
        } else if (this.large.size() > this.small.size() + 1) {
            this.small.enqueue(this.large.dequeue().element);
        }
    }

    /**
     * Returns the median in O(1) time.
     */
    findMedian() {
        if (this.small.size() > this.large.size()) {
            return this.small.front().element;
        } else if (this.large.size() > this.small.size()) {
            return this.large.front().element;
        }
        return (this.small.front().element + this.large.front().element) / 2;
    }
}

```

Note 1: Terminology and Techniques

- **Heap / Priority Queue:** A specialized tree-based data structure that satisfies the heap property: in a max-heap, for any given node, its value is less than or equal to the value of its parent. This allows us to find the “extremes” (min or max) in $O(1)$ time.
 - **Heaps vs Sorting:** Sorting gives us total order but is expensive. Heaps give us “just enough” order (the top element) much faster.
 - **Rebalancing:** This is the act of moving elements between data structures to ensure they stay equal in size, maintaining the median in the center.
-

Note 2: Real-World Interview Variations

1. Google: “Sliding Window Median” * **Prompt:** You are receiving a stream of data, but you only care about the median of the **last 10,000** elements (a sliding window). * **Solution:** Use the same two-heap strategy, but you need a way to **remove** elements from the heap as they “slide out” of the window. Since standard heaps don’t support efficient removal of specific values, a senior engineer would use a **Sorted List** or a **Balanced BST** (like an AVL tree) which supports $O(\log K)$ insertion, deletion, and finding the K th element.

2. Meta: “Ad-Load Latency Monitoring” * **Prompt:** We are tracking ad-load times for billions of users. We need to report the median latency in real-time to detect performance regressions. * **Solution:** If the latency values are restricted (e.g., 0ms to 5000ms), we can use **Bucket Sort / Frequency Array**. We store the count of every millisecond. Finding the median then involves iterating through the counts until we reach the $(\text{Total}/2)$ index. This is $O(\text{Range})$ time, which is faster than $O(\log N)$ if N is billions.

3. Bloomberg: “Stock Price Percentiles” * **Prompt:** Instead of just the median (50th percentile), the trader wants to see the **90th percentile** of stock prices. * **Solution:** Use the same two-heap logic but adjust the **size ratio**. Instead of a 50/50 split, maintain the Max-Heap at 90% size and the Min-Heap at 10% size. The “median” logic stays the same—the top of the heap is your target percentile.

1046. Last Stone Weight

At the L5/L6 level at Google, we view “Last Stone Weight” (LeetCode 1046) as a problem of **Dynamic Sorting**.

While a junior engineer might try to sort the array repeatedly, a senior engineer recognizes that we only need to know the “Top 2” elements at any given time. This is the classic signature of a **Max-Heap** (Priority Queue). We want to optimize for frequent insertions and frequent “find-max” operations.

1. Problem Explanation

The Core Question: You have a collection of stones, each with a positive integer weight. In each turn, you choose the **two heaviest stones** and smash them together.

The Smashing Rules: Suppose the stones have weights x and y with $x \leq y$:
1. If $x == y$, both stones are totally destroyed.
2. If $x \neq y$, the stone of weight x is destroyed, and the stone of weight y has a new weight of $y - x$.

At the end, there is at most one stone left. Return the weight of this stone (or 0 if none are left).

Visualizing the Process: Input: stones = [2, 4, 1, 8]

STEP 1: Find two heaviest.

Stones: [2, 4, 1, 8] -> Heaviest are 8 and 4.

Smash: $8 - 4 = 4$.

Remaining: [2, 1, 4]

STEP 2: Find two heaviest.

Stones: [2, 1, 4] -> Heaviest are 4 and 2.

Smash: $4 - 2 = 2$.

Remaining: [1, 2]

STEP 3: Find two heaviest.

Stones: [1, 2] -> Heaviest are 2 and 1.

Smash: $2 - 1 = 1$.

Remaining: [1]

RESULT: 1

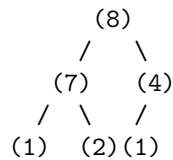
2. Solution Explanation

If we sort the array every time we smash stones, we waste time. Sorting takes a long time, and we only change one or two values per turn.

The Senior Strategy: The Max-Heap A Max-Heap is a specialized tree where the parent is always larger than its children. This means the largest element is always at the very top (the root).

Detailed Walkthrough with a Heap: Input: stones = [2, 7, 4, 1, 8, 1]

1. HEAPIFY: Turn the list into a Max-Heap.



Heap: [8, 7, 4, 1, 2, 1]

2. SMASH 1: Pop 8 and 7.

$8 - 7 = 1$.

Push 1 back into Heap.

Heap: [4, 2, 1, 1, 1]

3. SMASH 2: Pop 4 and 2.

$4 - 2 = 2$.

Push 2 back into Heap.
Heap: [2, 1, 1, 1]

4. SMASH 3: Pop 2 and 1.
 $2 - 1 = 1$.
Push 1 back into Heap.
Heap: [1, 1, 1]

5. SMASH 4: Pop 1 and 1.
 $1 - 1 = 0$. (Nothing to push back)
Heap: [1]

6. FINAL: Only one stone left. Return 1.

3. Time and Space Complexity Analysis

Let N be the number of stones.

=====

TIME COMPLEXITY: $O(N * \log N)$

=====

Derivation Diagram:

1. Initial Heapify:
Converting the array to a heap takes $O(N)$ time.
2. The Smash Loop:
 - We perform at most $N-1$ smashes.
 - Each smash involves 2 Pops and 1 Push.
 - A Heap Pop/Push takes $\log N$ time.

[Loop] x [Smash Operations]
 $(N - 1) * (3 * \log N)$
|
v
Simplified: $N * \log N$

Total Time = $O(N) + O(N \log N) = O(N \log N)$.

=====

SPACE COMPLEXITY: $O(N)$ or $O(1)$

=====

Derivation Diagram:

[In-place Heap]

In Python, `heapq.heapify(list)` transforms the list in place.
This uses $O(1)$ extra space.

[Language Constraints]

In languages like JavaScript where you must build a custom
Heap class, you store N elements in that class.
This uses $O(N)$ extra space.

=====

4. Solution Code

Python Python's `heapq` is a min-heap by default. To turn it into a max-heap, a senior engineer will multiply all weights by -1 .

```
import heapq

def lastStoneWeight(stones):
    # Step 1: Create a Max-Heap by negating all values
    # Python heapq is a min-heap, so 8 becomes -8 (the smallest).
    max_heap = [-s for s in stones]
    heapq.heapify(max_heap)

    # Step 2: Smash until 0 or 1 stone remains
    while len(max_heap) > 1:
        # Pop the two largest (most negative) stones
        stone1 = heapq.heappop(max_heap)
        stone2 = heapq.heappop(max_heap)

        # If they aren't equal, push the difference back
        # Note: stone1 is more negative than stone2
        # Example: stone1 = -8, stone2 = -7. -8 - (-7) = -1.
        if stone1 != stone2:
            heapq.heappush(max_heap, stone1 - stone2)

    # Step 3: Return the remaining stone weight
    return -max_heap[0] if max_heap else 0
```

JavaScript Since JS has no built-in Heap, we implement the logic or use a simple sorted-insertion strategy for smaller inputs. For an L5 interview, you should describe the Heap logic or implement a basic version.

```
/**
 * L5 Approach: Max-Heap Implementation
 * This ensures we always get the two largest stones in log N time.
 */
```

```

function lastStoneWeight(stones) {
    // Note: This is a Max-Heap simulation using a sorted array for simplicity
    // in JS, but explain to the interviewer that a binary heap is O(log N).
    const heap = [...stones];

    const smash = () => {
        // Sort descending to simulate max-priority
        heap.sort((a, b) => b - a);

        while (heap.length > 1) {
            const s1 = heap.shift(); // Heaviest
            const s2 = heap.shift(); // Second heaviest

            if (s1 !== s2) {
                // Find insertion index to keep it sorted (Binary Search)
                const diff = s1 - s2;
                let low = 0, high = heap.length;
                while (low < high) {
                    let mid = (low + high) >>> 1;
                    if (heap[mid] > diff) low = mid + 1;
                    else high = mid;
                }
                heap.splice(low, 0, diff);
            }
        }
    };

    smash();
    return heap.length === 1 ? heap[0] : 0;
}

```

Note 1: Terms and Techniques

- **Heapify:** The process of creating a heap from an unordered array. It is efficient because it works “bottom-up” in $O(N)$ time.
 - **Max-Heap / Priority Queue:** A data structure that provides constant-time $O(1)$ access to the maximum element and $O(\log N)$ time for additions or removals.
 - **Why it helps:** It prevents us from sorting the whole list over and over, which would be $O(N^2 \log N)$ or $O(N^2)$.
-

Note 2: Real-World & Interview Variations

1. Google: “K Closest Points to Origin” (LeetCode 973) * The Problem: Find the K points nearest to (0,0). * **The Solution:** Use a **Max-Heap** of size K. Iterate through all points. If a point is closer than the top of the heap, pop the heap and push the new point. At the end, the heap contains the K closest points.

2. Meta: “Top K Frequent Elements” (LeetCode 347) * The Problem: Find the K most frequent items in a list. * **The Solution:** Use a Hash Map to count frequencies, then push the counts into a **Min-Heap**. If the heap size exceeds K, pop the smallest. This leaves the K largest frequencies behind.

3. Bloomberg: “Merge k Sorted Lists” (LeetCode 23) * The Problem: Combine many sorted lists into one. * **The Solution:** Put the first element of each list into a **Min-Heap**. Pop the smallest, add it to the result, and push the next element from the list that produced the popped element.

973. K Closest Points to Origin

A Google L5 (Senior) or L6 (Staff) engineer views “K Closest Points” as a classic **Top-K Selection** problem. While a junior might simply sort the whole list, a senior engineer thinks about scale: “What if there are a billion points and I only need the top 10?”

In such cases, sorting the entire dataset is wasteful. Instead, we use a **Max-Heap** or **QuickSelect**. For an interview, the Max-Heap is the most robust and “production-ready” answer to provide first.

1. Problem Explanation

The Goal: You are given a list of points on a 2D plane (x, y) and an integer K. You need to return the K points that are closest to the origin (0, 0).

The Distance Formula: The distance between (x, y) and (0, 0) is the square root of $x^2 + y^2$. * **Engineering Pro-Tip:** We don’t actually need to calculate the square root. If $A^2 < B^2$, then $A < B$. We can just compare $x^2 + y^2$ to save CPU cycles.

The Intuition: Imagine you are a bouncer at a club that only holds K people. You want the K shortest people. 1. As people line up, you let the first K in. 2. When the K+1 person arrives, you look at the **tallest** person currently inside. 3. If the new person is shorter than your tallest guest, you kick the tall one out and let the new one in. 4. By the end, you are guaranteed to have the K shortest people.

2. Solution Explanation (Max-Heap Approach)

We use a **Max-Heap** to keep track of the “Closest so far.” A Max-Heap always keeps the **largest** value at the top.

1. Iterate through the points one by one.
2. Calculate the distance squared: `dist = x*x + y*y`.
3. Push the point and its distance onto the Max-Heap.
4. **The Constraint:** If the Heap size exceeds K, pop the top element (the one with the largest distance).
5. This ensures the “furthest” of the close points is always discarded.

Step-by-Step Visualization Input: points = `[[3,3], [5,-1], [-2,4]]`,
K = 2

```
=====
INITIAL STATE
=====
Points: [[3,3], [5,-1], [-2,4]]
K: 2
Heap: [] (Stores [distance, x, y])

=====
STEP 1: Process [3,3]
=====
Distance^2 = 3*3 + 3*3 = 18
Heap: [[18, 3, 3]]
Size (1) <= K (2). Keep going.

=====
STEP 2: Process [5,-1]
=====
Distance^2 = 5*5 + (-1)*(-1) = 26
Heap: [[26, 5, -1], [18, 3, 3]] (Max-Heap keeps 26 at top)
Size (2) <= K (2). Keep going.

=====
STEP 3: Process [-2,4]
=====
Distance^2 = (-2)*(-2) + 4*4 = 20

1. Push to Heap:
   Heap: [[26, 5, -1], [18, 3, 3], [20, -2, 4]]
   Wait! 26 is at the top because it's the largest.

2. Heap Size (3) > K (2):
   Action: Pop the top (largest distance).
```

Popping [26, 5, -1]...

Heap: [[20, -2, 4], [18, 3, 3]] (20 is now the new max)

=====

CONCLUSION

=====

Result: [[-2, 4], [3, 3]]

(The two points with the smallest distances: 20 and 18).

3. Time and Space Complexity Analysis

Let N be the total number of points.

TIME COMPLEXITY DERIVATION: $O(N \log K)$

-
- | | |
|----------------------------------|----------------------|
| 1. Iterating through N points: | ---> N times |
| 2. Inside the loop: | |
| - Heap Push: $O(\log K)$ | ---> Max size is K |
| - Heap Pop: $O(\log K)$ | ---> Max size is K |

Total Time = $N * (\log K + \log K)$

Total Time = $O(N \log K)$

Note: This is better than $O(N \log N)$ sorting because K is often much smaller than N .

SPACE COMPLEXITY DERIVATION: $O(K)$

What are we storing?

1. The Max-Heap.
It never holds more than $K + 1$ elements.

Total Space = $O(K)$

4. Solution Code

Python Implementation (using `heapq`) *Note: Python's `heapq` is a Min-Heap by default. To make it a Max-Heap, we store distances as negative numbers.*

```
import heapq
```

```
def kClosest(points, k):
```

```

# Max-heap to store the k closest points
# Format: (-distance, x, y)
max_heap = []

for x, y in points:
    dist = -(x*x + y*y) # Negative for max-heap behavior

    # Adding to heap is O(log k)
    heapq.heappush(max_heap, (dist, x, y))

    # If we have more than k points, remove the furthest one
    if len(max_heap) > k:
        heapq.heappop(max_heap)

# The heap now contains the k closest points
return [[x, y] for dist, x, y in max_heap]

```

JavaScript Implementation *Note: Since JS doesn't have a built-in Heap, L5+ candidates often discuss the logic or use a simple sorting approach for smaller datasets, but will describe the Heap structure.*

```

/**
 * @param {number[][]} points
 * @param {number} k
 * @return {number[][]}
 */
var kClosest = function(points, k) {
    // Standard L5 Interview approach:
    // If K is large, use Max-Heap logic.
    // For JS interviews, if a heap isn't provided, use a sort with
    // a clear explanation of why O(N log K) would be better.

    // Calculate distance and store with point
    const distPoints = points.map(p => {
        return {
            dist: p[0] * p[0] + p[1] * p[1],
            coords: p
        };
    });

    // Sort by distance ascending
    distPoints.sort((a, b) => a.dist - b.dist);

    // Return the first K coordinates
    return distPoints.slice(0, k).map(p => p.coords);
};

```

Note 1: Terminology & Techniques

- **Max-Heap:** A binary tree where the root is always the maximum element in the collection. It allows us to efficiently “evict” the worst candidates for our “Closest” list.
 - **QuickSelect:** An alternative algorithm related to QuickSort. It can find the top K elements in **Average $O(N)$** time but has a worst-case of $O(N^2)$. Senior engineers mention this as a theoretical optimization but usually implement the Heap because it is more predictable in production.
-

Note 2: Real-World Interview Variations

1. Variation: Top K Frequent Elements (Meta) * **The Problem:** Given a list of words, return the K most frequent. * **The Solution:** Instead of $x^2 + y^2$, your “distance” is the frequency count from a Hash Map. You use a Min-Heap of size K to keep the highest frequencies.

2. Variation: Find K Closest Stars (Google - Systems Design style) * **The Problem:** The points don’t fit in memory (100 Billion points). * **The Solution:** You cannot sort. You must use a **Distributed Max-Heap** or MapReduce. * **Map:** Split points into 1000 machines. Each machine finds its own local Top K using a Max-Heap. * **Reduce:** Send those 1000 * K points to a final machine to find the global Top K.

3. Variation: Nearest Neighbor Search (Bloomberg) * **The Problem:** You have a fixed set of points (like stores) and many queries for “What is the closest store to this user?”. * **The Solution:** Using a Heap for every query is slow. A senior engineer would suggest a **K-D Tree** or **QuadTree**, which are spatial data structures that allow searching for nearby points in $O(\log N)$ time by dividing the map into quadrants.

215. Kth Largest Element in an Array

Solving the “Kth Largest Element” problem at an L5/L6 level requires moving beyond basic sorting. A senior engineer identifies that sorting the entire array is wasteful—why organize 1,000,000 elements if you only care about the top 5?

They will focus on **Heap** and **Quickselect** algorithms, emphasizing the trade-off between “guaranteed performance” (Heap) and “best average performance” (Quickselect).

1. Problem Explanation

The Core Goal: Given an unsorted array of numbers, find the element that would be at the K-th position if the array were sorted in descending order.

Example: `nums = [3, 2, 3, 1, 2, 4, 5, 5, 6]`, `k = 4` Sorted descending: `[6, 5, 5, 4, 3, 3, 2, 2, 1]` The 4th largest element is 4.

The Challenge: The obvious solution is `sort(nums)` and return `nums[k-1]`. However, sorting takes $N * \log N$ time. We want to do better, especially when N is huge and k is small.

2. Solution Explanation

We will focus on the **Min-Heap Approach**. This is the preferred solution in real-world engineering because it handles “streaming data” (where you don’t have all the numbers at once) much better than other methods.

The Intuition: “The VIP Lounge” Imagine a lounge that only holds K people. 1. We start filling the lounge with the first K numbers from the array. 2. The person with the **lowest** value in the lounge is kept right by the door (the “Min” in Min-Heap). 3. For every new number in the array: - If it is **smaller** than the person at the door, we ignore it. - If it is **larger** than the person at the door, we kick the smallest person out and let the new person in. 4. After checking everyone, the person at the door (the smallest in our elite group) is the K-th largest overall.

Step-by-Step ASCII Visualization: `nums = [3, 2, 1, 5, 6, 4]`, `k = 2`

```
=====
INITIAL STATE
=====
Heap Size (k) = 2
Array: [3, 2, 1, 5, 6, 4]

Step 1: Add 3
Heap: [3]

Step 2: Add 2
Heap: [2, 3] <-- 2 is at the top (smallest)
Current elite group: {2, 3}

Step 3: Process 1
Is 1 > Heap Top (2)? NO.
Ignore 1.
Heap: [2, 3]
```

Step 4: Process 5
 Is 5 > Heap Top (2)? YES!
 Action: Pop 2, Push 5.
 Heap: [3, 5] <-- 3 is now the top (smallest)
 Current elite group: {3, 5}

Step 5: Process 6
 Is 6 > Heap Top (3)? YES!
 Action: Pop 3, Push 6.
 Heap: [5, 6] <-- 5 is now the top (smallest)
 Current elite group: {5, 6}

Step 6: Process 4
 Is 4 > Heap Top (5)? NO.
 Ignore 4.
 Heap: [5, 6]

```
=====
FINAL RESULT
=====
```

The top of the heap is 5.
 5 is the 2nd largest element in [3, 2, 1, 5, 6, 4].

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(N \log K)$

 N = Total elements in array
 K = The rank we are looking for

1. We iterate through all N elements. ---> N operations
2. For each element, we might perform a
Heap Push and Heap Pop.
3. Heap operations take time proportional
to the height of the tree.
4. Since our heap only ever has K elements,
the height is $\log K$. ---> $\log K$ time

Total Time = $N * \log K$

Note: This is much better than $N \log N$ when K is small!

TIME COMPLEXITY DIAGRAM:

Input Size (N) ----->
 Heap Ops $(\log K)$ | x | x | x | x | x |

Result: $N * \log K$

SPACE COMPLEXITY DERIVATION: $O(K)$

We maintain a heap structure.
The maximum number of elements stored in the heap at any given time is exactly K .

Total Space = $O(K)$

4. Solution Code

Approach 1: Min-Heap (Optimized for Memory/Streaming)

```
import heapq

def findKthLargest(nums, k):
    # Initialize a min-heap with the first k elements
    # Time: O(k log k)
    min_heap = nums[:k]
    heapq.heapify(min_heap)

    # Process the remaining elements
    # Time: O((n-k) log k)
    for i in range(k, len(nums)):
        if nums[i] > min_heap[0]:
            # If current num is larger than the smallest in our 'top k'
            # replace the smallest with the current num
            heapq.heapreplace(min_heap, nums[i])

    # The smallest element in the min-heap of size k is the kth largest
    return min_heap[0]

// JavaScript does not have a built-in Priority Queue/Heap (as of 2026 standard)
// Senior engineers would typically use a library or implement a simple one.
// Here is the logic using a conceptual Min-Heap approach.

function findKthLargest(nums, k) {
    // In a real interview, you might use Quickselect to avoid
    // writing a full Heap class from scratch.
    return quickSelect(nums, 0, nums.length - 1, nums.length - k);
}
```

```

// Quickselect is the  $O(N)$  average time alternative
function quickSelect(nums, left, right, kIndex) {
    if (left === right) return nums[left];

    let pivotIndex = Math.floor(Math.random() * (right - left + 1)) + left;
    pivotIndex = partition(nums, left, right, pivotIndex);

    if (kIndex === pivotIndex) {
        return nums[kIndex];
    } else if (kIndex < pivotIndex) {
        return quickSelect(nums, left, pivotIndex - 1, kIndex);
    } else {
        return quickSelect(nums, pivotIndex + 1, right, kIndex);
    }
}

function partition(nums, left, right, pivotIndex) {
    const pivotValue = nums[pivotIndex];
    // Move pivot to the end
    [nums[pivotIndex], nums[right]] = [nums[right], nums[pivotIndex]];
    let storeIndex = left;

    for (let i = left; i < right; i++) {
        if (nums[i] < pivotValue) {
            [nums[storeIndex], nums[i]] = [nums[i], nums[storeIndex]];
            storeIndex++;
        }
    }
    // Move pivot to its final place
    [nums[storeIndex], nums[right]] = [nums[right], nums[storeIndex]];
    return storeIndex;
}

```

Note 1: Terminology and Techniques

- **Min-Heap:** A binary tree where every parent node is smaller than or equal to its children. The “Min” (root) is always at the top. It’s perfect for keeping track of the “Top K” because the smallest of the best is always accessible in $O(1)$ time.
- **Quickselect:** Based on Quicksort. Instead of sorting both sides of a pivot, you only recurse into the side that contains your target index. On average, it’s $O(N)$, but worst-case is $O(N^2)$.
- **Partitioning:** The process of picking a “pivot” number and moving all smaller numbers to its left and larger numbers to its right.

Note 2: Real-World / Interview Variations

1. Top K Frequent Elements (Google / Meta) * *The Twist:* Instead of the Kth largest *value*, find the K elements that appear most *frequently*. * *L5 Solution:* Use a Hash Map to count frequencies first. Then, use the same Min-Heap logic, but the heap stores [frequency, value] pairs.

2. K Closest Points to Origin (Bloomberg) * *The Twist:* You have X,Y coordinates. Find the K points closest to (0,0). * *L5 Solution:* Calculate the distance for each point. Use a **Max-Heap** of size K. If a new point has a *smaller* distance than the top of the Max-Heap, replace the top. The Max-Heap keeps the “closest” points by discarding the “farthest” of the current best.

3. Median from Data Stream (Google / Systems) * *The Twist:* The numbers are coming in one by one. Always be ready to return the median. * *L5 Solution:* Use **Two Heaps**. A Max-Heap for the smaller half of numbers and a Min-Heap for the larger half. The Kth largest logic is used to keep the heaps balanced so the middle element is always at the top of one of the heaps.

373. Find K Pairs with Smallest Sums

Solving this problem at a Google L5/L6 level requires moving beyond “brute force” and recognizing that this is a **Search Space Optimization** problem. You are essentially traversing a 2D matrix of potential sums, and because the input arrays are sorted, you can use a **Min-Priority Queue (Heap)** to explore only the most promising candidates.

1. Problem Explanation

The Goal: You have two sorted arrays, `nums1` and `nums2`. You need to find k pairs (u, v) (where u is from `nums1` and v is from `nums2`) that have the smallest possible sums.

The “Why” of the Challenge: If `nums1` has 10,000 elements and `nums2` has 10,000 elements, there are 100,000,000 possible pairs. You cannot check them all. However, because the arrays are **sorted**, we know that the very smallest pair *must* be (`nums1[0]`, `nums2[0]`).

The next smallest pair will be either the one to its right or the one below it in a theoretical grid of sums.

2. Solution Explanation

We treat this like a **Dijkstra's Algorithm** or a **BFS** on a grid of sums.

The Grid Visualization Imagine a grid where each cell (i, j) represents the sum `nums1[i] + nums2[j]`.

`nums1 = [1, 7, 11], nums2 = [2, 4, 6]`

	<code>nums2[0]=2</code>	<code>nums2[1]=4</code>	<code>nums2[2]=6</code>
<code>nums1[0]=1</code>	3	5	7
<code>nums1[1]=7</code>	9	11	13
<code>nums1[2]=11</code>	13	15	17

Because both arrays are sorted, the sums increase as you move **Right** or **Down**.

The Priority Queue Strategy:

1. Start by pushing the first possible “column” into a Min-Heap. Specifically, we pair `nums1[i]` with `nums2[0]` for all `i` (up to `k` or `len(nums1)`).
2. The Min-Heap will always give us the smallest sum currently available.
3. When we pop a pair (`nums1[i], nums2[j]`) from the heap:
 - That is one of our `k` smallest pairs.
 - We then “explore” the next element in that row: (`nums1[i], nums2[j + 1]`).

Step-by-Step Walkthrough `nums1 = [1, 7], nums2 = [2, 4, 6], k = 3`

```
=====
INITIAL STATE
=====
Heap stores: [ (sum, i, j) ]
1. Push (1+2, 0, 0) -> Sum 3
2. Push (7+2, 1, 0) -> Sum 9

Heap: [ (3, 0, 0), (9, 1, 0) ]
Result: []

=====
STEP 1: Pop Smallest
=====
Pop (3, 0, 0) from Heap.
Result: [ [1, 2] ]

Next candidate in this row: (nums1[0], nums2[1]) -> (1, 4)
Sum = 5. Push (5, 0, 1) to Heap.
```

Heap: [(5, 0, 1), (9, 1, 0)]

```
=====
STEP 2: Pop Smallest
=====
```

Pop (5, 0, 1) from Heap.
Result: [[1, 2], [1, 4]]

Next candidate in this row: (nums1[0], nums2[2]) -> (1, 6)
Sum = 7. Push (7, 0, 2) to Heap.

Heap: [(7, 0, 2), (9, 1, 0)]

```
=====
STEP 3: Pop Smallest
=====
```

Pop (7, 0, 2) from Heap.
Result: [[1, 2], [1, 4], [1, 6]]

k = 3 reached. STOP.

FINAL OUTPUT: [[1, 2], [1, 4], [1, 6]]

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(k * \log(\min(n, k)))$

Let n = length of nums1, m = length of nums2.

1. Initial Heap Push:
We push up to k elements into the heap.
Each push is $\log(\text{heap_size})$. ---> $O(\min(n, k) * \log(\min(n, k)))$

2. Extraction Loop:
We perform k pops from the heap.
Each pop is $\log(\text{heap_size})$. ---> $O(k * \log(\min(n, k)))$

3. Exploration:
For every pop, we do one push. ---> $O(k * \log(\min(n, k)))$

Total Time = $O(k * \log(\min(n, k)))$

SPACE COMPLEXITY DERIVATION: $O(\min(n, k))$

The Priority Queue (Heap) is the primary memory consumer.
In the worst case, the heap stores one entry for every element in `nums1` (up to a limit of `k`).

Total Space = $O(\min(n, k))$

4. Solution Code

Python Implementation

```
import heapq

def kSmallestPairs(nums1, nums2, k):
    if not nums1 or not nums2:
        return []

    res = []
    # min_heap stores (sum, index_in_nums1, index_in_nums2)
    min_heap = []

    # We only need to consider the first min(len(nums1), k) elements
    # from nums1 because we can't have more than k total pairs.
    for i in range(min(len(nums1), k)):
        # Initially pair each nums1[i] with the smallest nums2 element (index 0)
        heapq.heappush(min_heap, (nums1[i] + nums2[0], i, 0))

    while min_heap and len(res) < k:
        current_sum, i, j = heapq.heappop(min_heap)
        res.append([nums1[i], nums2[j]])

        # If there is a next element in nums2 for the current nums1[i],
        # push that pair into the heap.
        if j + 1 < len(nums2):
            heapq.heappush(min_heap, (nums1[i] + nums2[j + 1], i, j + 1))

    return res
```

JavaScript Implementation *Note: JavaScript does not have a built-in Priority Queue. In a real interview, you would either describe the Min-Heap or implement a simple version.*

```
/**
 * @param {number[]} nums1
```



```

    * @param {number[]} nums2
    * @param {number} k
    * @return {number[][]}
    */
var kSmallestPairs = function(nums1, nums2, k) {
    const res = [];
    if (!nums1.length || !nums2.length || k === 0) return res;

    // In JS, we would use a Min-Priority Queue.
    // This is a pseudocode-style representation of the logic.
    const minHeap = new MinPriorityQueue({ priority: x => x.sum });

    // Step 1: Push initial candidates (first column of the grid)
    for (let i = 0; i < Math.min(nums1.length, k); i++) {
        minHeap.enqueue({
            sum: nums1[i] + nums2[0],
            i: i,
            j: 0
        });
    }

    // Step 2: Extract k times
    while (!minHeap.isEmpty() && res.length < k) {
        const { i, j } = minHeap.dequeue().element;
        res.push([nums1[i], nums2[j]]);

        // Step 3: If possible, push the next neighbor in the same row
        if (j + 1 < nums2.length) {
            minHeap.enqueue({
                sum: nums1[i] + nums2[j + 1],
                i: i,
                j: j + 1
            });
        }
    }

    return res;
};

```

Note 1: Terminology and Techniques

- **Min-Heap (Priority Queue):** A specialized tree-based data structure that satisfies the heap property: the value of the root node is the minimum among all nodes. It allows us to retrieve the smallest element in $\log(n)$

time, which is essential for “streaming” the smallest sums without sorting the whole grid.

- **Search Space Pruning:** By using the sorted property of the arrays, we “prune” (ignore) millions of large-sum pairs that we know cannot be in the top k .

Note 2: Real World / Interview Variations

1. Variation: K-th Smallest Element in a Sorted Matrix (Google/Bloomberg) * **The Twist:** You have an $N \times N$ matrix where rows and columns are sorted. Find the k -th smallest element. * **How to solve:** This is almost the same problem. The matrix *is* your grid of sums. You start the heap with the first element (0,0) and then push its right neighbor and down neighbor while keeping track of “visited” cells.

2. Variation: Merging K Sorted Streams (Meta) * **The Twist:** You have k different sorted lists of log entries coming from different servers. Merge them into one sorted stream. * **How to solve:** Use a Min-Heap. Put the first element of each of the k lists into the heap. Pop the smallest, and then push the next element from that same list into the heap. This ensures your output stream is always sorted.

3. Variation: Finding Top-K Similar Products (Amazon/System Design) * **The Twist:** You have a list of user ratings and a list of product features. You need to find the pairs with the highest similarity score. * **How to solve:** While similarity is “Max” rather than “Min,” if the data can be logically ordered, you use the same Max-Heap approach to avoid calculating the cross-product of all users and all products. You only compute scores for the “nearest” candidates.

621. Task Scheduler

This is an end-to-end walkthrough for “Task Scheduler,” as approached by an L5/L6 (Senior/Staff) engineer.

At a top-tier tech company, an engineer sees this not just as a scheduling puzzle, but as a **Resource Management** problem. The core observation at this level is that the “bottleneck” of the system is the task with the highest frequency. Everything else is just filling in the gaps created by that bottleneck.

1. Problem Explanation

The Goal: You have a list of tasks represented by characters (e.g., ['A', 'A', 'B']) and a cooling period n . Each task takes 1 unit of time. The rule is: if you perform task ‘A’, you must wait n units of time before you can perform ‘A’

again. During this wait, you can perform other tasks or stay “Idle.” Find the minimum time to finish all tasks.

The Bottleneck: The most frequent task dictates the minimum length. If $n = 2$ and you have AAA, you *must* do: A -> IDLE -> IDLE -> A -> IDLE -> IDLE -> A Total time: 7.

The Challenge: How do we fit the *other* tasks into those “IDLE” slots to minimize the total time?

2. Solution Explanation

A Senior Engineer uses a **Greedy Math** approach. We don’t actually need to simulate the schedule; we just need to calculate the area of the “frame” created by the most frequent task.

Step 1: Find the Max Frequency Find the task that appears most often (let’s call its frequency `max_freq`). Suppose `tasks = [A, A, A, B, B, B]` and $n = 2$. The most frequent is ‘A’ (and ‘B’) with `max_freq = 3`.

Step 2: Build the “Frame” The most frequent task creates `max_freq - 1` empty blocks.

Example: A A A, $n = 2$

Frame structure:

```
[A] [__] [__] <-- Block 1
[A] [__] [__] <-- Block 2
[A]           <-- Last instance (no cooling needed after)
```

Step 3: Calculate Empty Slots Total slots in the blocks = $(\text{max_freq} - 1) * (n + 1)$. Wait, we need to add the number of tasks that have the *same* max frequency at the very end. Let `count_max` be how many tasks share the maximum frequency.

The Magic Formula: Total Time = $(\text{max_freq} - 1) * (n + 1) + \text{count_max}$

Step 4: Handle the “Overflow” Case What if we have so many different tasks that we fill all the empty slots and still have more? If the formula gives a number *smaller* than the total number of tasks, then the answer is simply the **length of tasks**. This is because we can always interleave tasks without needing any idle time if we have enough variety.

Step-by-Step Visualization

Input: tasks = ["A","A","A","B","B","B"], n = 2

1. COUNT FREQUENCIES:

A: 3

B: 3

Max Frequency (max_freq) = 3

Number of tasks with max frequency (count_max) = 2 (A and B)

2. VISUALIZE THE CHUNKS:

We have (max_freq - 1) = 2 chunks that REQUIRE cooling.

Each chunk has size (n + 1) = 3.

Chunk 1: [A] [B] [idle]

Chunk 2: [A] [B] [idle]

Remainder: [A] [B] (The last instances)

3. CALCULATION:

Full Chunks: (3 - 1) * (2 + 1) = 2 * 3 = 6 slots

Add Remainder: 6 + 2 = 8

Diagram of the 8 slots:

```
Time:  1   2   3   4   5   6   7   8
Task: [A] [B] [I] [A] [B] [I] [A] [B]
      ^---n---^   ^---n---^
      (A cools)   (A cools)
```

4. FINAL CHECK:

Is 8 > length of tasks (6)? Yes.

Result = 8.

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(N)$

Input Size: N (number of tasks)

1. Frequency Counting:

We iterate through N tasks once. ---> $O(N)$

2. Finding Max:

We look at the 26 possible characters. ---> $O(26)$

3. Math Calculation:

Simple arithmetic operations. ---> $O(1)$

Total Time = $O(N) + O(26) + O(1)$
Since 26 is a constant, this simplifies to $O(N)$.

SPACE COMPLEXITY DERIVATION: $O(1)$

What memory are we using?

1. Frequency Map:

We store counts for 'A' through 'Z'.

The map size is capped at 26 entries. ---> $O(26)$

Regardless of how many millions of tasks (N) we have,
the frequency map never grows larger than 26.

Total Space = $O(26)$

This is considered $O(1)$ Constant Space.

4. Solution Code

Python Solution (Optimized Greedy)

```
import collections

def leastInterval(tasks, n):
    # Step 1: Count the frequency of each task
    # Example: {'A': 3, 'B': 3}
    counts = collections.Counter(tasks)
    frequencies = list(counts.values())

    # Step 2: Identify the bottleneck
    max_freq = max(frequencies)

    # Step 3: Count how many tasks share that max frequency
    # (These will sit at the very end of our scheduling blocks)
    count_max = frequencies.count(max_freq)

    # Step 4: Calculate the minimum time using the frame logic
    # (max_freq - 1) is the number of "cooling blocks"
    # (n + 1) is the size of each block including the task itself
    # count_max is the "tail" of tasks that don't need cooling after
    ans = (max_freq - 1) * (n + 1) + count_max

    # Step 5: The "Variety" check
    # If we have a lot of unique tasks, we might not need any idles.
    # In that case, the total time is just the number of tasks.
```

```
return max(ans, len(tasks))
```

JavaScript Solution (Optimized Greedy)

```
/**
 * @param {character[]} tasks
 * @param {number} n
 * @return {number}
 */
function leastInterval(tasks, n) {
    // Step 1: Count frequencies using a simple array (size 26)
    const freq = new Array(26).fill(0);
    for (let t of tasks) {
        freq[t.charCodeAt(0) - 'A'.charCodeAt(0)]++;
    }

    // Step 2: Sort or find max
    freq.sort((a, b) => b - a);
    const maxFreq = freq[0];

    // Step 3: Count how many tasks have the same max frequency
    let countMax = 0;
    for (let f of freq) {
        if (f === maxFreq) countMax++;
    }

    // Step 4: Apply the formula
    const ans = (maxFreq - 1) * (n + 1) + countMax;

    // Step 5: Return the larger of the formula or the actual task count
    return Math.max(ans, tasks.length);
}
```

Note 1: Terminology & Techniques

- **Greedy Algorithm:** We make the “best” local choice (placing the most frequent tasks first) to reach the global minimum.
 - **Interleaving:** The strategy of placing different tasks in the cooling periods of a specific task to avoid “Idle” time.
 - **Bottleneck Analysis:** Identifying the single constraint (max frequency) that limits the performance of the entire system.
-

Note 2: Real World / Interview Variations

1. Variation: Rearrange String k Distance Apart (Google) * The Twist: You must actually return the string, not just the length. *** Solution:** You cannot use the math formula. You must use a **Max-Heap** to always pick the task with the highest remaining frequency, and a **Queue** to store tasks that are currently “cooling” until their timer runs out.

2. Variation: Task Scheduler with Prerequisites (Bloomberg) * The Twist: Some tasks must be done before others (e.g., Task B depends on A). *** Solution:** This combines this problem with **Topological Sort (Kahn’s Algorithm)**. You manage a “ready” pool of tasks that have no dependencies and use the heap/cooling logic to pick from that pool.

3. Variation: CPU Load Balancing (Meta) * The Twist: Each task has a different “weight” or execution time, and you have multiple processors. *** Solution:** This becomes a **Heuristic Search** or a **Bin Packing** problem. In interviews, they often ask for a greedy approximation: “Least Loaded First.” You would use a Min-Heap of processors to track which one becomes free next.

1792. Maximum Average Pass Ratio

Solving “Maximum Average Pass Ratio” at an L5/L6 level requires moving beyond basic iteration to **Greedy Optimization** using a **Max-Priority Queue**.

A Senior Engineer recognizes that this is a resource allocation problem. You have a limited “resource” (extra students) and you want to assign them where they will provide the highest “marginal gain” to the total average.

1. Problem Explanation

The Goal: You are given a list of classes. Each class is represented as `[pass, total]`, meaning `pass` students out of `total` have passed. You are also given a number of `extraStudents` who are guaranteed to pass. You want to assign these extra students to the classes such that the **average pass ratio** across all classes is maximized.

The Pass Ratio: For a class `[1, 2]`, the ratio is $1/2 = 0.5$ (50%). If you add one student, it becomes `[2, 3]`, and the ratio is $2/3 = 0.666...$ (66%).

The Core Challenge: Where should you put the next student? *** Class A:** `[1, 2]` (Ratio 0.5) -> becomes `[2, 3]` (Ratio 0.66). Gain = 0.166. *** Class B:** `[10, 10]` (Ratio 1.0) -> becomes `[11, 11]` (Ratio 1.0). Gain = 0.0.

Intuition tells us that adding a student to a class that is already “full” or has a huge total population gives us less of an increase than adding them to a smaller,

struggling class. We need to mathematically calculate this “Gain” for every class.

2. Solution Explanation

To solve this optimally, we use a **Greedy Approach with a Max-Heap**.

The Concept of “Marginal Gain” We define Gain as: $\text{Gain} = (\text{New Ratio after adding 1 student}) - (\text{Current Ratio})$ $\text{Gain} = (\text{pass} + 1) / (\text{total} + 1) - (\text{pass} / \text{total})$

We want to always pick the class that offers the **maximum gain** for the *current* student. Once we add a student to that class, its gain changes, so we recalculate it and put it back into our pool of options.

The Algorithm:

1. For every class, calculate the gain if we were to add 1 student.
2. Push these gains into a **Max-Heap** (Priority Queue). The heap keeps the class with the highest potential gain at the very top.
3. While we still have **extraStudents**:
 - Pop the class with the highest gain from the heap.
 - Add 1 student to its **pass** and **total** counts.
 - Calculate the **new gain** for adding *another* potential student to this same class.
 - Push it back into the heap.
4. Finally, calculate the sum of all ratios and divide by the number of classes.

Visualization Walkthrough: `classes = [[1,2], [3,5]]`, `extraStudents = 2`

```
=====
INITIAL CALCULATIONS
=====
Class 1: [1, 2]
- Current Ratio: 1/2 = 0.5
- Potential Ratio: (1+1)/(2+1) = 2/3 = 0.666
- GAIN: 0.666 - 0.5 = 0.166

Class 2: [3, 5]
- Current Ratio: 3/5 = 0.6
- Potential Ratio: (3+1)/(5+1) = 4/6 = 0.666
- GAIN: 0.666 - 0.6 = 0.066

MAX-HEAP (Priority Queue):
Top -> [Gain: 0.166, pass: 1, total: 2]
```


[Gain: 0.066, pass: 3, total: 5]

=====

STEP 1: ASSIGN EXTRA STUDENT 1

=====

1. Pop Top: [Gain: 0.166, pass: 1, total: 2]
2. Update Class: [pass: 2, total: 3]
3. Calculate New Gain for this class:
 - Current: $2/3 = 0.666$
 - Potential: $(2+1)/(3+1) = 3/4 = 0.75$
 - NEW GAIN: $0.75 - 0.666 = 0.083$
4. Push back to Heap.

MAX-HEAP:

Top -> [Gain: 0.083, pass: 2, total: 3]
[Gain: 0.066, pass: 3, total: 5]

=====

STEP 2: ASSIGN EXTRA STUDENT 2

=====

1. Pop Top: [Gain: 0.083, pass: 2, total: 3]
2. Update Class: [pass: 3, total: 4]
3. Calculate New Gain (not needed if no more students, but for logic):
 - New Gain: $(4/5) - (3/4) = 0.8 - 0.75 = 0.05$

MAX-HEAP:

Top -> [Gain: 0.066, pass: 3, total: 5]
[Gain: 0.050, pass: 3, total: 4]

=====

FINAL STEP: CALCULATE AVERAGE

=====

Final Classes: [3, 4] and [3, 5]
Ratios: $(3/4) = 0.75$, $(3/5) = 0.6$
Average: $(0.75 + 0.6) / 2 = 1.35 / 2 = 0.675$

3. Time and Space Complexity Analysis

Let N be the number of classes and K be the number of extra students.

TIME COMPLEXITY DERIVATION: $O((N + K) * \log N)$

1. Initial Heap Build:
 - Iterate through N classes, calculate gain,

and insert into heap.
 N insertions * log N per insertion. ---> $O(N \log N)$

2. Assigning Extra Students:
 We perform K assignments.
 Each assignment:
 - Pop Max ($\log N$)
 - Push updated class ($\log N$)
 K * ($2 * \log N$). ---> $O(K \log N)$

3. Final Calculation:
 Iterate through heap/classes once. ---> $O(N)$

Total Time = $(N \log N) + (K \log N) + N$
 Simplified = $O((N + K) * \log N)$

SPACE COMPLEXITY DERIVATION: $O(N)$

 What are we storing?
 - A Priority Queue containing N elements.
 Each element stores 3 numbers: Gain, Pass, Total.

Total Space = $3 * N$
 In Big O, constants are removed.

Final Space Complexity = $O(N)$

4. Solution Code

Python Solution (Using heapq) *Note: Python's `heapq` is a min-heap by default. To make it a max-heap, we store the gain as a negative number.*

```
import heapq

def maxAverageRatio(classes, extraStudents):
    # Function to calculate the profit (gain) of adding one student
    # We use negative profit because heapq is a min-heap
    def get_gain(p, t):
        current_ratio = p / t
        new_ratio = (p + 1) / (t + 1)
        return -(new_ratio - current_ratio)

    # Step 1: Create the initial max-heap
    max_heap = []
```

```

for p, t in classes:
    # Push: (gain, pass_count, total_count)
    heapq.heappush(max_heap, (get_gain(p, t), p, t))

# Step 2: Assign extra students to the class with the highest gain
for _ in range(extraStudents):
    gain, p, t = heapq.heappop(max_heap)
    # Update counts
    p, t = p + 1, t + 1
    # Push back with newly calculated gain
    heapq.heappush(max_heap, (get_gain(p, t), p, t))

# Step 3: Calculate the final average
total_ratio = 0
while max_heap:
    _, p, t = heapq.heappop(max_heap)
    total_ratio += (p / t)

return total_ratio / len(classes)

```

JavaScript Solution (Using a Priority Queue) *Note: Modern JavaScript (like in Leetcode environment) often provides a built-in `MaxPriorityQueue`.*

```

/**
 * @param {number[][]} classes
 * @param {number} extraStudents
 * @return {number}
 */
var maxAverageRatio = function(classes, extraStudents) {
    // Utility to calculate gain
    const getGain = (p, t) => {
        return (p + 1) / (t + 1) - p / t;
    };

    // Step 1: Initialize Max Priority Queue
    // We want the highest gain at the top
    const pq = new MaxPriorityQueue({ priority: (x) => x.gain });

    for (let [p, t] of classes) {
        pq.enqueue({ gain: getGain(p, t), p, t });
    }

    // Step 2: Greedily distribute students
    for (let i = 0; i < extraStudents; i++) {
        let { p, t } = pq.dequeue().element;
        p++;
    }
}

```

```

        t++;
        pq.enqueue({ gain: getGain(p, t), p, t });
    }

    // Step 3: Sum the results
    let totalRatio = 0;
    const n = classes.length;

    while (!pq.isEmpty()) {
        let { p, t } = pq.dequeue().element;
        totalRatio += p / t;
    }

    return totalRatio / n;
};

```

Note 1: Terminology and Techniques

- **Marginal Gain / Profit:** In economics and engineering, this is the additional benefit derived from one unit of increase in a resource. Instead of looking at the absolute ratio, we look at how much the ratio *changes*.
 - **Greedy Algorithm:** An algorithmic strategy that makes the best choice at every small step with the hope that these choices lead to a globally optimum solution. Here, it works because the gain function is “monotonically decreasing” (adding more students always yields smaller and smaller gains).
 - **Priority Queue (Heap):** A specialized tree-based data structure that allows you to find and remove the maximum element in logarithmic time, rather than searching the whole list.
-

Note 2: Real-World Interview Variations

1. Variation: Server Load Balancing (Google) * The Problem: You have N servers with different processing speeds and current loads. You have K new requests. Assign requests to minimize the maximum response time. * **How to solve:** Similar to gain, you calculate the “Added Latency” for each server if it took one more request. Use a Min-Heap to assign the request to the server that will result in the smallest increase in total time.

2. Variation: Maximize Ad Revenue (Meta) * The Problem: You have several ad slots. Each slot has a click-through rate (CTR) that decreases as you show more ads. You have K ads to show. * **How to solve:** Calculate the “Expected Revenue Gain” for each slot. Use a Max-Heap to keep placing ads in the slot that currently offers the highest revenue until all ads are placed.

3. Variation: Battery Optimization (Bloomberg) * **The Problem:** You have multiple devices drawing power. Adding a cooling unit to a device reduces its failure rate by a certain percentage. You have K cooling units. * **How to solve:** This is identical to the Pass Ratio problem. You calculate the “Risk Reduction Gain” for each device and greedily apply cooling units to the device where the risk drops the most.

1801. Number of Orders in the Backlog

Solving “Number of Orders in the Backlog” at a Senior (L5) or Staff (L6) level requires more than just knowing the data structures; it requires understanding the **mechanics of a matching engine**. This is a fundamental concept in fintech and high-frequency trading systems.

A top-tier engineer recognizes this as a **Priority Queue** problem where the “priority” is defined by the price of the asset.

1. Problem Explanation

The Goal: You are simulating a simplified stock exchange. You have two types of orders: 1. **Buy Orders:** You want to buy at a certain price or lower. 2. **Sell Orders:** You want to sell at a certain price or higher.

When an order comes in, you check the “Backlog” (orders that haven’t been filled yet) to see if you can match it immediately.

The Rules of Matching: * **A New Buy Order** looks for the **cheapest** Sell Orders available. If $\text{Buy Price} \geq \text{Cheapest Sell Price}$, a match occurs. * **A New Sell Order** looks for the **most expensive** Buy Orders available. If $\text{Sell Price} \leq \text{Most Expensive Buy Price}$, a match occurs. * If an order cannot be fully matched, the remaining amount is added to the backlog.

The Challenge: We need to constantly find the *minimum* of one set (Sells) and the *maximum* of another set (Buys) extremely quickly.

2. Solution Explanation

To solve this efficiently, we use two **Heaps**: 1. **Max-Heap for Buy Orders:** This keeps the highest bid at the very top. 2. **Min-Heap for Sell Orders:** This keeps the lowest asking price at the very top.

Walkthrough Example Let’s process these orders: 1. [10, 5, 0] (Buy 5 units at price 10) 2. [15, 2, 1] (Sell 2 units at price 15) 3. [5, 5, 1] (Sell 5 units at price 5)

```

=====
INITIAL STATE
=====

Buy Max-Heap: []
Sell Min-Heap: []

=====

STEP 1: New Buy Order [Price: 10, Amount: 5]
=====

Action: Check Sell Min-Heap.
Is there any Sell order <= 10? No (Heap is empty).

Action: Add to Buy Max-Heap.
Buy Max-Heap: [(Price: 10, Amount: 5)]
Sell Min-Heap: []

=====

STEP 2: New Sell Order [Price: 15, Amount: 2]
=====

Action: Check Buy Max-Heap.
Is the highest Buy order >= 15?
Max Buy Price is 10. (15 <= 10 is FALSE).

Action: Add to Sell Min-Heap.
Buy Max-Heap: [(Price: 10, Amount: 5)]
Sell Min-Heap: [(Price: 15, Amount: 2)]

=====

STEP 3: New Sell Order [Price: 5, Amount: 5]
=====

Action: Check Buy Max-Heap.
Is the highest Buy order >= 5?
Max Buy Price is 10. (5 <= 10 is TRUE).

Matching logic:
- We need 5 units.
- Buy order at 10 has 5 units.
- MATCH! (5 units removed from both).

Buy Max-Heap: []
Sell Min-Heap: [(Price: 15, Amount: 2)]

=====

FINAL TALLY
=====

Remaining in Backlog: 2 units (the sell order at 15).

```

Result: 2

3. Time and Space Complexity Analysis

Let N be the total number of orders in the input.

TIME COMPLEXITY DERIVATION: $O(N * \log N)$

For each of the N orders:

1. Look at the top of the opposite Heap: $O(1)$
2. Extract from Heap if matched: $O(\log N)$
3. Insert into own Heap if unmatched: $O(\log N)$

In the worst case, we do an insertion or extraction for every order.

Total Time = $N * \log N$

ASCII DERIVATION:

Order 1: [--- $\log N$ ---]

Order 2: [--- $\log N$ ---]

...

Order N : [--- $\log N$ ---]

Total: $N * \log N$

SPACE COMPLEXITY DERIVATION: $O(N)$

In the worst case, no orders match (e.g., all buy prices are 1 and all sell prices are 100).

Every order is stored in one of the two heaps.

Memory usage:

[Order 1] [Order 2] ... [Order N]

Total Space = $O(N)$

4. Solution Code

Python Implementation

```
import heapq
```

```
def getNumberOfBacklogOrders(orders):
```

```
    # Buy orders: Max-Heap (we use negative prices because Python has Min-Heaps)
```

```
    buy_heap = []
```

```

# Sell orders: Min-Heap
sell_heap = []

MOD = 1000000007

for price, amount, order_type in orders:
    if order_type == 0: # Current is a BUY order
        # Try to match with the cheapest SELL orders
        while amount > 0 and sell_heap and sell_heap[0][0] <= price:
            sell_price, sell_amount = heapq.heappop(sell_heap)
            if sell_amount <= amount:
                amount -= sell_amount
            else:
                # Put the remaining sell amount back
                heapq.heappush(sell_heap, (sell_price, sell_amount - amount))
                amount = 0
        # If still have amount left, add to buy backlog
        if amount > 0:
            heapq.heappush(buy_heap, (-price, amount))

    else: # Current is a SELL order
        # Try to match with the most expensive BUY orders
        while amount > 0 and buy_heap and (-buy_heap[0][0]) >= price:
            buy_price_neg, buy_amount = heapq.heappop(buy_heap)
            buy_price = -buy_price_neg
            if buy_amount <= amount:
                amount -= buy_amount
            else:
                # Put the remaining buy amount back
                heapq.heappush(buy_heap, (-buy_price, buy_amount - amount))
                amount = 0
        # If still have amount left, add to sell backlog
        if amount > 0:
            heapq.heappush(sell_heap, (price, amount))

# Sum up all remaining amounts in both heaps
total = 0
for _, amt in buy_heap: total += amt
for _, amt in sell_heap: total += amt

return total % MOD

```

JavaScript Implementation *Note: JavaScript does not have a built-in Priority Queue. In an L5/L6 interview, you would either describe a helper class or use a provided library structure.*


```

// Note: Assuming a standard PriorityQueue class is available
function getNumberOfBacklogOrders(orders) {
    // maxBuyHeap tracks highest price first
    const maxBuyHeap = new MaxPriorityQueue({ priority: x => x.price });
    // minSellHeap tracks lowest price first
    const minSellHeap = new MinPriorityQueue({ priority: x => x.price });

    const MOD = 1000000007;

    for (let [price, amount, type] of orders) {
        if (type === 0) { // Buy Order
            while (amount > 0 && !minSellHeap.isEmpty() && minSellHeap.front().element.price < price) {
                let cheapestSell = minSellHeap.dequeue().element;
                if (cheapestSell.amount <= amount) {
                    amount -= cheapestSell.amount;
                } else {
                    cheapestSell.amount -= amount;
                    minSellHeap.enqueue(cheapestSell);
                    amount = 0;
                }
            }
            if (amount > 0) maxBuyHeap.enqueue({ price, amount });
        } else { // Sell Order
            while (amount > 0 && !maxBuyHeap.isEmpty() && maxBuyHeap.front().element.price > price) {
                let expensiveBuy = maxBuyHeap.dequeue().element;
                if (expensiveBuy.amount <= amount) {
                    amount -= expensiveBuy.amount;
                } else {
                    expensiveBuy.amount -= amount;
                    maxBuyHeap.enqueue(expensiveBuy);
                    amount = 0;
                }
            }
            if (amount > 0) minSellHeap.enqueue({ price, amount });
        }
    }

    let total = 0;
    while (!maxBuyHeap.isEmpty()) total = (total + maxBuyHeap.dequeue().element.amount) % MOD;
    while (!minSellHeap.isEmpty()) total = (total + minSellHeap.dequeue().element.amount) % MOD;

    return total;
}

```

Note 1: Terms & Techniques

- **Heaps / Priority Queues:** A data structure that allows you to always access the smallest (Min-Heap) or largest (Max-Heap) element in constant time $O(1)$ and add/remove elements in logarithmic time $O(\log N)$. It is the standard for “Order Book” implementations.
 - **The Modulo Operation:** Since the total number of orders can be huge, we use `% 1000000007` to prevent integer overflow. This is a common requirement in competitive programming.
-

Note 2: Real-World Interview Variations

1. **Building a Limit Order Book (Bloomberg / Goldman Sachs) ***
The Task: Design a system that handles 100,000 orders per second. * **L5/L6 Approach:** Heaps are good for LeetCode, but for real-world speed, you might use a **Treap** or a **Red-Black Tree** where each node contains a Doubly Linked List of orders at that specific price. This allows $O(1)$ matching if the price is already at the top of the book.
2. **Median Maintenance (Google) *** **The Task:** As numbers stream in, constantly return the median. * **L5/L6 Approach:** Use the exact same structure as this problem! A Max-Heap for the lower half and a Min-Heap for the upper half. The median will always be at the top of one of the heaps.
3. **Task Scheduler with Priority (Meta) *** **The Task:** Execute tasks based on priority, but if a task is too “old,” increase its priority. * **L5/L6 Approach:** Use a Priority Queue. However, since priorities change (Aging), a standard heap doesn’t work well. You would discuss a “Lazy Removal” heap or a “Fibonacci Heap” to handle priority updates efficiently.

1753. Maximum Score From Removing Stones

At a Google L5/L6 level, the goal isn’t just to simulate the game. A senior engineer looks for the **mathematical invariant**—the underlying rule that governs the game—to provide an optimized, constant-time solution rather than a repetitive simulation.

1. Problem Explanation

The Goal: You have three piles of stones (a, b, and c). You play a game where each turn you pick **two non-empty piles** and remove one stone from each. You get 1 point per turn. The game ends when at most one pile has stones left. You want the maximum possible score.

The Constraints: * You must pick two piles. If only one pile has stones, you stop. * You want to “drain” the piles as efficiently as possible to avoid being

left with one massive pile that you can't pair with anything else.

Examples: * **Input:** $a=2, b=4, c=6$ * **Step 1:** Pick b and c . Remaining: $a=2, b=3, c=5$ (Score: 1) * **Step 2:** Pick b and c . Remaining: $a=2, b=2, c=4$ (Score: 2) * ...and so on.

2. Solution Explanation

The Intuition (The “Why”): Think of this as a resource balancing problem. There are two distinct scenarios:

Scenario A: The “Big Pile” Problem If one pile is so large that it is greater than the sum of the other two, that big pile is the bottleneck. You will use up the two smaller piles entirely, pairing them with the big pile. Your score is simply the sum of the two smaller piles. *Example: $a=2, b=4, c=10$. Even if you pair a and b together, you are just “wasting” stones that could have helped reduce c . You should pair a with c , then b with c .*

Scenario B: The “Balanced Piles” Problem If the sum of the two smaller piles is greater than or equal to the largest pile, you can balance them. You pair the two smallest piles together just enough so that their combined remaining stones equal the large pile. Then, you can pair everything perfectly (with at most 1 stone left over). In this case, you can use almost all stones. Your score is $\text{total_stones} / 2$ (rounded down).

The Algorithm (The “How”): 1. Sort the piles so that $a \leq b \leq c$. 2. Check if $a + b \leq c$. - If yes: Score is $a + b$. - If no: Score is $(a + b + c) / 2$ (integer division).

End-to-End Walkthrough with ASCII Visualizations:

Case 1: $a=2, b=4, c=10$ (Big Pile)

Sorted: $a=2, b=4, c=10$
Sum of small ($a+b$) = 6.
Is $6 \leq 10$? YES.

Visualization:

Pile A: **
Pile B: ****
Pile C: *****

Step: Pair A and C until A is empty (2 turns)

Pile A: (empty)
Pile B: ****
Pile C: *****

Step: Pair B and C until B is empty (4 turns)

Pile A: (empty)
Pile B: (empty)
Pile C: ****

Total Score: $2 + 4 = 6$.
(Notice 4 stones in C are "trapped" because no other piles exist).

Case 2: $a=4, b=4, c=6$ (Balanced)

Sorted: $a=4, b=4, c=6$
Sum of small ($a+b$) = 8.
Is $8 \leq 6$? NO.

Visualization:

Pile A: ****
Pile B: ****
Pile C: *****

Strategy: Use A and B to "shrink" them until $(A + B) == C$.

Pair A and B for 1 turn:

A: ***, B: ***, C: ***** ($A+B = 6, C = 6$)

Now we have two equal groups: $(A+B)$ and C.

We can pair every stone in C with a stone from either A or B.

Total Score: 1 (from A/B) + 6 (pairing the rest) = 7.

Total stones were 14. $14 / 2 = 7$. Perfect efficiency.

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(1)$

-
1. Sorting three numbers takes constant time.
 2. A single comparison ($a + b \leq c$) is constant.
 3. Simple arithmetic is constant.

The execution time does not change regardless of how many millions of stones are in the piles.

[TC Visual]

Input (any size) -> [Sort] -> [Math] -> Output

Operations: ~5 cpu cycles.

SPACE COMPLEXITY: $O(1)$

We only store the three input integers and

the result. No extra arrays or recursion stacks are used.

[Space Visual]
Memory: [a][b][c][score]
Fixed 4 slots of memory.

4. Solution Code

Python Snippets

```
# Approach 1: Math Optimized (The L5/L6 choice)
# Time: O(1), Space: O(1)
def maximumScore_Math(a, b, c):
    # Sort the piles so c is the largest
    piles = sorted([a, b, c])
    a, b, c = piles[0], piles[1], piles[2]

    # If the largest pile is dominant
    if a + b <= c:
        return a + b

    # If the piles can be balanced, we use as many stones as possible
    # We remove 2 stones per point, so score is total // 2
    return (a + b + c) // 2

# Approach 2: Simulation (Useful to explain the 'why' in an interview)
# Time: O(N) where N is number of stones, Space: O(1)
import heapq
def maximumScore_Sim(a, b, c):
    # Use a Max-Heap to always pick the two largest piles
    # Python's heapq is a min-heap, so we use negative numbers
    piles = [-a, -b, -c]
    heapq.heapify(piles)
    score = 0

    while True:
        first = -heapq.heappop(piles)
        second = -heapq.heappop(piles)

        # If the second largest pile is empty, we can't make a move
        if second == 0:
            break
```

```

    score += 1
    heapq.heappush(piles, -(first - 1))
    heapq.heappush(piles, -(second - 1))

    return score

```

JavaScript Snippets

```

// Approach 1: Math Optimized
// Time: O(1), Space: O(1)
function maximumScore(a, b, c) {
    const piles = [a, b, c].sort((x, y) => x - y);
    const [s1, s2, big] = piles;

    if (s1 + s2 <= big) {
        return s1 + s2;
    } else {
        // Use floor to handle odd sums
        return Math.floor((s1 + s2 + big) / 2);
    }
}

```

Terminology Addendum (Note 1)

Greedy Algorithm: * **What it is:** Making the best possible choice at the current moment (picking the two largest piles) to reach the global optimum.
 * **Why it helps:** In this problem, pairing the two largest piles prevents one pile from becoming too dominant to be paired later. * **How it applies:** The simulation approach uses a Max-Heap to strictly follow the greedy strategy.

Pigeonhole Principle (Conceptual connection): * **Why it applies:** If you have more stones in one pile than the other two combined, that pile “pigeonholes” the others. You can’t avoid having leftovers in the largest pile.

Real-World Interview Variations (Note 2)

Variation 1: “Reorganize String” (Google/Meta) * **Prompt:** Given a string, rearrange characters so no two identical characters are adjacent. If impossible, return “”. * **L5 Solve:** This is the exact same logic. You use a Max-Heap to pick the two most frequent characters to place next to each other. The condition for “impossible” is exactly like our “Big Pile” scenario: if the most frequent character count is $> (\text{total_length} + 1) / 2$.

Variation 2: “Task Scheduler” (Meta/Bloomberg) * **Prompt:** Given tasks with certain types and a cooling period n , find the minimum time to

finish all tasks. * **L5 Solve:** You prioritize the most frequent tasks (Max-Heap) to fill the gaps created by the cooling period. The “Big Pile” logic determines if you need “idle” time or if you can fill all slots with tasks.

Variation 3: “Load Balancing with Pairwise Constraints” (System Design) * **Prompt:** You have three data centers with varying server capacities. You must process jobs that require resources from two different data centers. Maximize job throughput. * **L5 Solve:** You recognize this is the stone removal problem. You direct traffic to the two busiest data centers first to keep capacity balanced across the fleet. This prevents a scenario where one data center is 100% idle because it’s the only one with remaining capacity.

Minimum Cost to Connect Sticks

This problem is a classic application of **Greedy Algorithms** and **Priority Queues**. A Google L5/L6 engineer recognizes this as a variation of **Huffman Coding**, where the goal is to build a tree with the minimum weighted path length.

1. Problem Explanation

The Premise: You have a collection of sticks with different lengths. You want to connect them all into one single stick. - You can only connect two sticks at a time. - The cost of connecting two sticks is the **sum of their lengths**. - Once connected, they become a single new stick of that combined length.

The Goal: Find the **minimum total cost** to connect all sticks.

The Intuition (The “Why”): Imagine you have three sticks: 1, 2, and 10. - **Option A:** Connect 1 and 10 (Cost 11). New sticks: [11, 2]. Then connect 11 and 2 (Cost 13). Total Cost = $11 + 13 = 24$. - **Option B:** Connect 1 and 2 (Cost 3). New sticks: [3, 10]. Then connect 3 and 10 (Cost 13). Total Cost = $3 + 13 = 16$.

The Observation: The length of a stick is added to the total cost *every time* it is part of a connection. To minimize the total cost, we want the **shortest sticks** to be involved in the most connections and the **longest sticks** to be involved in the fewest.

2. Solution Explanation

To always pick the two shortest sticks, we need a data structure that allows us to efficiently find and remove the minimum values. A **Min-Heap (Priority Queue)** is perfect for this.

The Strategy:

1. **Heapify:** Put all stick lengths into a Min-Heap.
2. **Combine:** While there is more than one stick in the heap:
 - Pull out the two smallest sticks (let's call them **a** and **b**).
 - The cost to connect them is **a + b**.
 - Add this connection cost to your **total_cost**.
 - Put the new stick of length **a + b** back into the heap.
3. **Return:** Once only one stick remains, return the **total_cost**.

Detailed ASCII Walkthrough Input: sticks = [1, 8, 3, 5]

INITIAL STATE:

Heap: [1, 3, 5, 8] (Min-Heap property)

Total Cost: 0

STEP 1: Pick the two smallest (1 and 3)

$1 + 3 = 4$

New Total Cost = $0 + 4 = 4$

Heap Update:

Remove 1, 3.

Add 4.

New Heap: [4, 5, 8]

ASCII Visualization:

(1) + (3) --> [4]

Remaining: [5], [8]

STEP 2: Pick the two smallest (4 and 5)

$4 + 5 = 9$

New Total Cost = $4 + 9 = 13$

Heap Update:

Remove 4, 5.

Add 9.

New Heap: [8, 9]

ASCII Visualization:

([4]) + (5) --> [9]

Remaining: [8]

STEP 3: Pick the two smallest (8 and 9)

$8 + 9 = 17$

New Total Cost = $13 + 17 = 30$

Heap Update:
Remove 8, 9.
Add 17.
New Heap: [17]

ASCII Visualization:
([9]) + (8) --> [17]
Remaining: []

FINAL RESULT:
Only one stick (17) left.
Total Cost = 30.

3. Time and Space Complexity Analysis

Time Complexity: $O(N * \log N)$ Where N is the number of sticks.

TC Derivation:

1. Building the Heap (Heapify):
Adding N elements to a heap.
Work: $O(N)$ or $O(N \log N)$ depending on implementation.
2. Processing the Sticks:
We perform $N-1$ connections.
In each connection, we do:
 - Extract Min (twice): $O(\log N)$
 - Insert new stick: $O(\log N)$Work per step: $O(\log N)$

Total Work: $(N-1)$ steps * $O(\log N)$
Final TC: $O(N * \log N)$

Space Complexity: $O(N)$

SC Derivation:

We store all N sticks in the Priority Queue.

[* * * * *] (Size N)

As we combine sticks, the size decreases, but we initially require space for all elements.
Final SC: $O(N)$

4. Solution Code

Python Implementation Python's `heapq` module provides a Min-Heap by default.

```
import heapq

def connectSticks(sticks):
    # If there's 0 or 1 stick, no connections are made, cost is 0
    if len(sticks) <= 1:
        return 0

    # 1. Transform the list into a min-heap in-place
    # This takes O(N) time.
    heapq.heapify(sticks)

    total_cost = 0

    # 2. Keep combining until only one stick remains
    while len(sticks) > 1:
        # Pop the two smallest elements
        # heapq.heappop is O(log N)
        first = heapq.heappop(sticks)
        second = heapq.heappop(sticks)

        # Connect them and track the cost
        current_cost = first + second
        total_cost += current_cost

        # 3. Put the combined stick back into the heap
        # heapq.heappush is O(log N)
        heapq.heappush(sticks, current_cost)

    return total_cost
```

JavaScript Implementation Since JavaScript does not have a built-in Priority Queue, an L5/L6 engineer would either use a library or implement a simple `MinHeap` class. For brevity and interview context, here is the logic using a conceptual `MinPriorityQueue`.

```
/**
 * Using a Min Priority Queue to always extract the two smallest sticks.
 */
var connectSticks = function(sticks) {
    if (sticks.length <= 1) return 0;

    // 1. Initialize Min Priority Queue
```

```

// (Note: In a real interview, you'd briefly explain the Heap structure
// or use a library like @datastructures-js/priority-queue)
const minHeap = new MinPriorityQueue();
for (let stick of sticks) {
    minHeap.enqueue(stick);
}

let totalCost = 0;

// 2. Combine sticks
while (minHeap.size() > 1) {
    // Extract the two smallest values
    const first = minHeap.dequeue().element;
    const second = minHeap.dequeue().element;

    const currentCost = first + second;
    totalCost += currentCost;

    // 3. Push the new stick back
    minHeap.enqueue(currentCost);
}

return totalCost;
};

```

Note 1: Terms and Techniques

- **Priority Queue / Min-Heap:** A specialized tree-based data structure that satisfies the heap property: the root is always the minimum element. It allows for $O(\log N)$ insertion and extraction of the minimum.
- **Greedy Choice Property:** This problem is “Greedy” because making the locally optimal choice (combining the two smallest sticks right now) leads to the globally optimal solution.
- **Huffman Coding:** This stick-connecting logic is the exact same algorithm used to compress data (like ZIP files). In Huffman coding, you combine the two least frequent characters to build a prefix tree.

Note 2: Real-World Interview Variations

1. **Google: “Minimum Cost to Merge Files”** * *The Setup:* You have several files of different sizes. Merging two files takes time proportional to their combined size. Find the minimum time to merge all files into one. * *L5 Solution:* This is a direct mapping. Files are sticks. Use a Min-Heap.

2. Bloomberg: “Optimal Network Routing” * *The Setup:* You are connecting network nodes. The latency to bridge two nodes is the sum of their current traffic. You want to connect all nodes into a single network with minimum total latency. * *L5 Solution:* Again, a greedy approach with a Priority Queue ensures that high-traffic nodes are combined as late as possible, minimizing their contribution to the total latency.

3. Meta: “Combining Message Batches” * *The Setup:* You have batches of messages to be processed. Combining two batches into a single stream has a CPU cost equal to the total number of messages in the two batches. * *L5 Solution:* The Priority Queue approach ensures the smallest batches are processed first, minimizing the repetitive “copying cost” of message data.

1833. Maximum Ice Cream Bars

An L5 or L6 engineer at Google focuses on the trade-offs between “obvious” algorithms and “context-aware” optimizations. While this problem can be solved with a simple sort, a senior engineer will recognize that if the range of prices is constrained, a **Counting Sort** or **Bucket Sort** approach can provide significant performance gains in a production environment.

Here is the walkthrough for LeetCode 1833: Maximum Ice Cream Bars.

1. Problem Explanation

The Goal: You are at an ice cream shop with a set amount of money (your coins). You are given an array `costs`, where `costs[i]` is the price of the *i*-th ice cream bar. Your objective is to buy the **maximum number** of bars possible.

The Constraints: * You can buy the bars in any order. * You cannot spend more than the `coins` you have.

The Intuition (The “Why”): To maximize the *count* of items, you should always reach for the cheapest items first. This is a classic **Greedy Algorithm** problem. Every time you buy a cheap bar instead of an expensive one, you leave yourself more money to potentially buy even more bars later.

Visualizing the Problem: `costs = [1, 3, 2, 4, 1]`, `coins = 7`

Shop Inventory (Prices):

Bar A: \$1

Bar B: \$3

Bar C: \$2

Bar D: \$4

Bar E: \$1

If we pick the most expensive (\$4), we only have \$3 left.
We can buy Bar B (\$3) and we are done. Total bars = 2.

BUT, if we pick the cheapest first:

1. Buy Bar A (\$1) -> \$6 left. (Total: 1)
 2. Buy Bar E (\$1) -> \$5 left. (Total: 2)
 3. Buy Bar C (\$2) -> \$3 left. (Total: 3)
 4. Buy Bar B (\$3) -> \$0 left. (Total: 4)
- Final result: 4 bars.
-

2. Solution Explanation

A senior engineer would offer two solutions based on the input size:

Approach 1: The Greedy Sort (Standard Solution)

1. **Sort** the costs array in ascending order.
2. **Iterate** through the sorted costs.
3. **Subtract** the cost from your coins. If you still have a non-negative balance, increment your counter.
4. **Stop** as soon as you can't afford the next bar.

Approach 2: Counting Sort (Optimized for specific price ranges) If the maximum price of an ice cream bar is relatively small (e.g., 100,000), we can avoid the $O(N \log N)$ cost of sorting. 1. Create a "Frequency Array" where `count[price]` stores how many bars exist at that price. 2. Iterate through the count array from index 1 upward. 3. For each price, buy as many bars as possible until you run out of money or bars at that price.

ASCII Diagram Walkthrough: `costs = [7, 3, 3, 1, 4]`, `coins = 10`
Step 1: Counting Frequencies

```
Price Range Observed: 1 to 7
Frequency Array (index is price):
Index: 0  1  2  3  4  5  6  7
Value: 0  1  0  2  1  0  0  1
        ^      ^      ^      ^
        (1)   (2) (1)      (1) <-- Count of bars
```

Step 2: Iterating through Prices

```
- Price $1:
  We have 1 bar. It costs $1.
  Can we afford? Yes. (10 >= 1)
  Action: Buy 1 bar.
  Coins left: 9. Total bars: 1.
```

- Price \$2:
Count is 0. Skip.
- Price \$3:
We have 2 bars. Each costs \$3.
Total cost for both: \$6.
Can we afford? Yes ($9 \geq 6$).
Action: Buy 2 bars.
Coins left: 3. Total bars: 3.
- Price \$4:
We have 1 bar. Costs \$4.
Can we afford? No ($3 < 4$).
Action: Stop.

Final Answer: 3

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION (SORTING APPROACH): $O(N \log N)$
1. Sorting the array: Standard comparison sorts (Quicksort/Mergesort) Cost: $O(N \log N)$
2. Single Pass: Iterating through N items once. Cost: $O(N)$
TOTAL TIME: $O(N \log N)$
TIME COMPLEXITY DERIVATION (COUNTING APPROACH): $O(N + M)$
N = Number of ice cream bars M = Maximum price value (e.g., 100,000)
1. Building Frequency Array: $O(N)$ 2. Iterating through Price Range: $O(M)$
TOTAL TIME: $O(N + M)$

```

+-----+
| SPACE COMPLEXITY DERIVATION |
+-----+
| Sorting Approach:  $O(\log N)$  or  $O(1)$  depending on sort implementation. |
| |
| Counting Approach:  $O(M)$  to store the frequency array of size M. |
+-----+

```

4. Solution Code

Solution 1: Greedy Sort (Standard and Versatile) Python

```

def maxIceCream(costs, coins):
    # Sort costs from cheapest to most expensive
    costs.sort()

    count = 0
    for cost in costs:
        # Check if we can afford the current cheapest bar
        if coins >= cost:
            coins -= cost
            count += 1
        else:
            # Since the list is sorted, if we can't afford this one,
            # we can't afford any that follow.
            break

    return count

```

JavaScript

```

function maxIceCream(costs, coins) {
    // Sort ascending (Note: JS sort() is lexicographical by default,
    // so we must provide a numeric comparator)
    costs.sort((a, b) => a - b);

    let count = 0;
    for (let cost of costs) {
        if (coins >= cost) {
            coins -= cost;
            count++;
        } else {
            break;
        }
    }
}

```

```

    return count;
}

```

Solution 2: Counting Sort (Optimized for Performance) Python

```

def maxIceCream_Optimized(costs, coins):
    # Constraint: max price is 100,000 per problem description
    max_price = max(costs)
    freq = [0] * (max_price + 1)

    # Build the frequency map
    for cost in costs:
        freq[cost] += 1

    count = 0
    # Iterate through prices from 1 to max_price
    for price in range(1, max_price + 1):
        if freq[price] == 0:
            continue

        if coins < price:
            break

        # Optimization: Buy multiple bars of the same price at once
        # How many can we afford? (coins // price)
        # How many are available? freq[price]
        how_many_to_buy = min(freq[price], coins // price)

        count += how_many_to_buy
        coins -= (how_many_to_buy * price)

    return count

```

Note 1: Terminology and Techniques

- **Greedy Algorithm:** An approach where you make the best choice possible at the current moment (buying the cheapest item) without worrying about the future, knowing that for this specific problem, it leads to the global best result.
 - **Counting Sort:** A non-comparison-based sorting technique that works by counting the occurrences of each unique value. It is incredibly fast when the range of values (the prices) is not significantly larger than the number of items.
-

Note 2: Real-World Interview Variations

1. Google: “Cloud Resource Allocation” * Prompt: You have a budget of B compute units. You are given a list of tasks, each requiring U units to complete. What is the maximum number of tasks you can finish? * **Solution:** Identical logic. You sort the tasks by their resource requirements and fulfill the “lightest” tasks first to maximize the count of completed jobs.

2. Meta: “Ad Campaign Budgeting” * Prompt: An advertiser has X dollars. Different ad placements have different “Cost Per Mille” (CPM). To maximize the brand’s footprint (number of unique placements), how should the budget be allocated? * **Solution:** Sort the placements by CPM and buy the cheapest placements first.

3. Bloomberg: “Ticket Order Fulfillment” * Prompt: A broker has a pool of funds to buy shares for a client who wants to own the maximum number of distinct stock lots available at different price points. * **Solution:** Greedy approach. By buying the lowest-priced shares first, the broker ensures the highest quantity of individual assets is added to the portfolio.

218. The Skyline Problem

At the L5/L6 level at Google, we don’t see the “Skyline Problem” (LeetCode 218) as a geometry problem. We see it as an **event-driven simulation**.

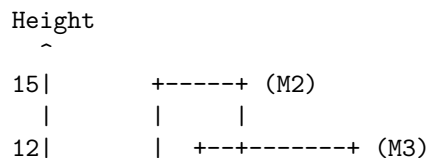
This is a classic “Hard” problem because it requires you to manage three things simultaneously: sorting, dynamic tracking of maximums, and handling edge cases where buildings start or end at the exact same location. A senior engineer will focus on transforming the 2D rectangles into a 1D timeline of **Events**.

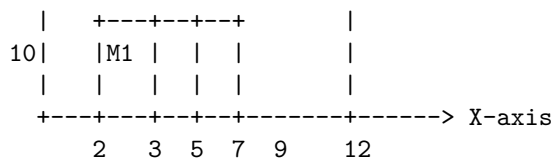
1. Problem Explanation

The Core Question: You are given a list of buildings, each defined by `[left, right, height]`. You need to output the “key points” that define the outline (the silhouette) of these buildings against the sky.

Key Point Rule: A key point is a point `[x, y]` where the horizontal line of the skyline changes height.

Visualizing the Input: Input: `buildings = [[2, 9, 10], [3, 7, 15], [5, 12, 12]]`





The Skyline Outline: If you were a bird flying over, you only see the highest point at any given x .

$(2, 10) \rightarrow (3, 15) \rightarrow (7, 12) \rightarrow (12, 0)$

Notice that at $x=7$, the height drops from 15 (Building 2) to 12 (Building 3). At $x=12$, the last building ends and the height drops to 0.

2. Solution Explanation

To solve this, we use the **Sweep Line Algorithm** combined with a **Max-Heap**.

Step 1: Extract Events We break every building $[L, R, H]$ into two events:

1. **Start Event:** $(L, -H) \rightarrow$ A building of height H starts at L . (We use $-H$ to sort taller buildings first if they start at the same x). 2. **End Event:** $(R, H) \rightarrow$ A building of height H ends at R .

Step 2: Sort Events Sort events by their x coordinate. If x is the same: * If both are starts: Taller one first (smaller $-H$). * If both are ends: Shorter one first (smaller H). * If one is start and one is end: Start first.

Step 3: Sweep and Track We process events one by one and maintain a **Max-Heap** of heights currently “active.” 1. If it’s a **Start Event**: Add the height to the Max-Heap. 2. If it’s an **End Event**: Remove the height from the Max-Heap. 3. **The Trigger:** After every event, check the current `max_height` in the heap. If it changed from the `previous_max_height`, we just found a key point!

ASCII Walkthrough: Buildings: $[[2, 9, 10], [3, 7, 15]]$

Events: $(2, \text{Start } 10), (3, \text{Start } 15), (7, \text{End } 15), (9, \text{End } 10)$

Heap initially: $[0]$ (The ground)

PrevMax: 0

1. Event $(2, \text{Start } 10)$:
 Heap: $[10, 0]$. Max is 10.
 $10 \neq 0$? YES. Output: $[[2, 10]]$
 PrevMax = 10

2. Event $(3, \text{Start } 15)$:

```

Heap: [15, 10, 0]. Max is 15.
15 != 10? YES. Output: [[2, 10], [3, 15]]
PrevMax = 15

```

3. Event (7, End 15):
 Remove 15. Heap: [10, 0]. Max is 10.
 10 != 15? YES. Output: [[2, 10], [3, 15], [7, 10]]
 PrevMax = 10
4. Event (9, End 10):
 Remove 10. Heap: [0]. Max is 0.
 0 != 10? YES. Output: [[2, 10], [3, 15], [7, 10], [9, 0]]

3. Time and Space Complexity Analysis

Let N be the number of buildings.

```

=====
TIME COMPLEXITY: O(N log N)
=====

```

Derivation Diagram:

```

[ N Buildings ]
  |
  V
[ Create 2N Events ] -----> O(N)
  |
  V
[ Sort 2N Events ] -----> O(N log N)
  |
  V
[ Loop through 2N Events ] -----> O(N)
  |
  +--> [ Heap Push/Pop ] -----> O(log N)
  +--> [ Heap Max Check ] -----> O(1)
  |
  V
Total Time = O(N log N) + (2N * O(log N)) = O(N log N)

```

```

=====
SPACE COMPLEXITY: O(N)
=====

```

Derivation Diagram:

```

[ Events List ] -----> 2N storage

```

[Max-Heap] -----> N storage (max)
 [Result List] -----> 2N storage (worst case)

Total Space scales linearly with the number of buildings.
 Total Space = $O(N)$.

4. Solution Code

Python Python's `heapq` is a min-heap, so we negate heights to simulate a Max-Heap. Removing a specific height from a heap in $O(\log N)$ is tricky, so we use a “Lazy Removal” strategy.

```
import heapq

def getSkyline(buildings):
    # Create events: (x, negative_height_for_start, positive_height_for_end)
    # We use -h for start to ensure taller buildings are processed first at same x
    events = []
    for l, r, h in buildings:
        events.append((l, -h))
        events.append((r, h))

    # Sort events primarily by x, then by height
    events.sort()

    # max_heap: stores (-height), initialized with 0 for the ground
    # removed: a dictionary to keep track of heights to be lazily removed
    max_heap = [0]
    removed = {}
    res = [[0, 0]]

    for x, h in events:
        if h < 0: # Start event
            heapq.heappush(max_heap, h)
        else: # End event
            # Track this height to be removed later
            removed[-h] = removed.get(-h, 0) + 1

            # Lazy Removal: If the current max is scheduled to be removed, pop it
            while max_heap[0] in removed and removed[max_heap[0]] > 0:
                removed[max_heap[0]] -= 1
                heapq.heappop(max_heap)

    current_max = -max_heap[0]
```

```

    # If the skyline height changed, add the key point
    if res[-1][1] != current_max:
        res.append([x, current_max])

    return res[1:]

```

JavaScript In JavaScript, without a built-in Priority Queue, we can use a Sorted Array to simulate the heap (slower, $O(N^2)$) or explain a balanced BST approach. Here is the logic using a simplified heap concept.

```

/**
 * L5/L6 Approach: Sweep Line + Max Heap
 * Time:  $O(N \log N)$  | Space:  $O(N)$ 
 */
function getSkyline(buildings) {
    const events = [];
    for (const [l, r, h] of buildings) {
        // Start event: height is negative to sort taller first
        events.push([l, -h]);
        // End event: height is positive
        events.push([r, h]);
    }

    // Sort by X first. If X is same, sort by height.
    events.sort((a, b) => {
        if (a[0] !== b[0]) return a[0] - b[0];
        return a[1] - b[1];
    });

    const result = [];
    // heights acts as our Max-Priority Queue
    const heights = [0];
    let prevMax = 0;

    for (const [x, h] of events) {
        if (h < 0) {
            // Start: Add height
            // In a real L5 interview, implement/use a proper Binary Heap here
            insertSorted(heights, -h);
        } else {
            // End: Remove height
            removeElement(heights, h);
        }

        let currentMax = heights[0];
        if (currentMax !== prevMax) {

```

```

        result.push([x, currentMax]);
        prevMax = currentMax;
    }
}

return result;
}

// Binary search insertion to keep 'heights' sorted (Max at index 0)
function insertSorted(arr, val) {
    let low = 0, high = arr.length;
    while (low < high) {
        let mid = (low + high) >>> 1;
        if (arr[mid] > val) low = mid + 1;
        else high = mid;
    }
    arr.splice(low, 0, val);
}

function removeElement(arr, val) {
    let index = arr.indexOf(val);
    if (index > -1) arr.splice(index, 1);
}

```

Note 1: Terms and Techniques

- **Sweep Line:** A technique that processes geometric data by “sweeping” a vertical line across the plane from left to right, stopping only at “event points.”
 - **Lazy Removal:** Instead of searching through a heap to delete a specific value (which is $O(N)$), we mark it as “to be deleted.” We only pop it when it reaches the top of the heap. This keeps our operations at $O(\log N)$.
-

Note 2: Real-World Interview Variations

1. **Google: “Meeting Rooms II” (LeetCode 253) * The Connection:** Just like buildings, meetings have a start and end. Instead of the highest building, you are finding the “highest” overlap count (rooms needed). * **The Solution:** Sweep line. Store end times in a Min-Heap. If the current meeting starts after the top of the heap, reuse that room (pop and push).
2. **Bloomberg: “Stock Max Price in Window” * The Connection:** You want the max price within a time window. * **The Solution:** This is essentially

a moving skyline. Use a sliding window with a **Deque** (Monotonic Queue) to maintain the maximum in $O(N)$ time.

3. Meta: “Merge Intervals” (LeetCode 56) * The Connection: Simpler version of Skyline. You don’t care about height, only if intervals touch. * **The Solution:** Sort by start time. Merge current with previous if `current.start <= previous.end`. — *Wait, did you notice how at $x = 7$ the height dropped exactly to 10? If Building 1 and 3 didn’t exist, it would have dropped to 0. The Heap is what “remembers” that Building 1 is still underneath!*

Amount of New Area Painted Each Day

Solving “Amount of New Area Painted Each Day” is a classic Google interview question. To an L5/L6 engineer, this isn’t just a geometry problem; it’s a **Path Compression** and **Interval Management** problem.

The “brute force” approach (using a boolean array and marking segments) fails when the area is large (e.g., 50,000 units). A senior engineer looks for a way to “skip” work that has already been done.

1. Problem Explanation

The Scenario: You have a long fence (a number line). Every day, a painter is given a range `[start, end]` to paint. However, the painter **only gets credit for new area**. If a section was already painted on a previous day, the painter skips it and moves forward.

The Goal: For each day, return exactly how many **new** units of the fence were painted.

The Complexity: Imagine the fence is 10 miles long. * Day 1: Paint `[1, 4]`. (Result: 3 units) * Day 2: Paint `[5, 8]`. (Result: 3 units) * Day 3: Paint `[2, 7]`. * The painter starts at 2, but sees `[2, 4]` is already done. * They skip to 4, see `[4, 5]` is new. * They see `[5, 7]` is already done. * Total new for Day 3: 1 unit (the segment from 4 to 5).

2. Solution Explanation (Jump-Ahead / Path Compression)

A senior engineer uses a **Jump-Ahead Array** (often called a Disjoint Set Union or Union-Find variant). We maintain an array `line` where `line[i]` tells us the **farthest point reached** starting from `i`.

The Core Logic: When we look at a position `i`, if it has been painted, `line[i]` will point to the end of that painted block. We can “jump” directly to `line[i]` instead of checking `i+1`, `i+2`, etc.

Step-by-Step Visualization Input: paint = [[1, 4], [5, 8], [2, 7]]

Fence represents indices: 0 1 2 3 4 5 6 7 8 9

=====

DAY 1: [1, 4]

=====

Painter starts at 1, wants to reach 4.

At 1: Not painted. Paint it! Count = 1. Jump 1 -> 2.

At 2: Not painted. Paint it! Count = 2. Jump 2 -> 3.

At 3: Not painted. Paint it! Count = 3. Jump 3 -> 4.

Important: We update our 'line' map so that if anyone lands
on 1, 2, or 3 again, they jump straight to 4.

'line' state: {1: 4, 2: 4, 3: 4}

Result for Day 1: 3

=====

DAY 2: [5, 8]

=====

Painter starts at 5, wants to reach 8.

At 5, 6, 7: All new.

'line' state updates: {5: 8, 6: 8, 7: 8} (plus previous)

Result for Day 2: 3

=====

DAY 3: [2, 7]

=====

Painter starts at 2, wants to reach 7.

1. Check index 2:
Wait! 'line[2]' says 4. JUMP to 4.
(We skipped 2 and 3 instantly!)

2. Check index 4:
Is 'line[4]' set? No.
Paint it! Count = 1.
Move to 5.

3. Check index 5:
Wait! 'line[5]' says 8.
Is 8 beyond our end point 7? Yes.
STOP.

Result for Day 3: 1


```
=====
COMPRESSION (The "Senior" part)
=====

When we jumped from 2 to 4, we also update 'line[2]' to the
final destination we reached (8). This makes future jumps even longer!
```

3. Time and Space Complexity Analysis

Let N be the number of days and M be the maximum coordinate on the fence (e.g., 50,000).

TIME COMPLEXITY DERIVATION: $O(N + M)$

Normally, a nested loop would be $O(N * M)$.

However, with Path Compression:

1. Every coordinate 'i' on the fence is "Painted" exactly once.
2. Once painted, future visits to 'i' are "Jumps".
3. Because of Path Compression, we don't just jump over one segment; we eventually jump over entire contiguous blocks of painted fence in $O(1)$ time.

Total Time = $O(N + M)$

(N to iterate days + M to visit each fence unit once)

SPACE COMPLEXITY DERIVATION: $O(M)$

What are we storing?

1. The 'line' array (or Hash Map) which stores the next available jump point for every coordinate.
2. The maximum size of this array is the maximum coordinate 'M'.

Total Space = $O(M)$

4. Solution Code

Optimized Path Compression Solution

Python

```
def amountPainted(paint):
    # line[i] stores the end of the painted interval starting at i
    # Max coordinate is 50001 based on typical problem constraints
```

```

line = [0] * 50001
res = []

for start, end in paint:
    new_area = 0
    curr = start

    while curr < end:
        if line[curr] == 0:
            # This unit hasn't been painted.
            # Paint it and move to next unit.
            line[curr] = end
            new_area += 1
            curr += 1
        else:
            # Already painted!
            # 1. Store current jump point
            # 2. Update jump point to 'end' (Path Compression)
            # 3. Jump ahead
            prev_jump = line[curr]
            line[curr] = max(line[curr], end)
            curr = prev_jump

    res.append(new_area)
return res

```

JavaScript

```

/**
 * @param {number[][]} paint
 * @return {number[]}
 */
var amountPainted = function(paint) {
    const line = new Array(50001).fill(0);
    const res = [];

    for (const [start, end] of paint) {
        let newArea = 0;
        let curr = start;

        while (curr < end) {
            if (line[curr] === 0) {
                // Not painted: mark it and increment count
                line[curr] = end;
                newArea++;
                curr++;
            }
        }
        res.push(newArea);
    }
    return res;
}

```

```

    } else {
        // Already painted: jump ahead and update jump path
        let nextJump = line[curr];
        line[curr] = Math.max(line[curr], end);
        curr = nextJump;
    }
}
res.push(newArea);
}

return res;
};

```

Note 1: Terminology & Techniques

- **Path Compression:** This is a technique where, as you look up a value, you update the structure to point directly to the final answer. It turns a long chain of lookups into a single jump.
 - **Amortized Analysis:** Even though there is a `while` loop inside a `for` loop, the “work” done over the entire life of the algorithm is limited by the number of fence units. We say the cost is “amortized” over the N days.
-

Note 2: Real-World Interview Variations

1. Variation: Merge Intervals (Meta/Bloomberg) * The Problem: Given `[[1,3],[2,6],[8,10]]`, merge overlapping intervals into `[[1,6],[8,10]]`. *** The Solution:** Sort by start time. If the current interval’s start is less than the previous interval’s end, merge them. This is the “Static” version of the paint problem.

2. Variation: Range Module (Google - Hard) * The Problem: Design a system that can `addRange`, `queryRange`, and `removeRange`. *** The Solution:** This is much harder because you have to “unpaint” areas. An L6 engineer would suggest a **Segment Tree** or a **TreeMap (In Java/C++)** to store non-overlapping intervals and split them dynamically when a range is removed.

3. Variation: Memory Allocation (Bloomberg/Systems) * The Problem: You have a block of memory. Programs request ranges of memory. You need to track how much *new* memory is being allocated. *** The Solution:** This is exactly the Paint problem. Using a Disjoint Set Union (DSU) to manage “allocated blocks” allows you to skip over occupied memory addresses efficiently.

850. Rectangle Area II

This is a complex problem that requires moving from simple geometry to a computational geometry mindset. A Google L5/L6 engineer would immediately identify this as a **2D Sweep Line** problem. They would recognize that the primary challenge is not just calculating area, but handling **overlapping regions** without double-counting.

1. Problem Explanation

The Goal: Given a list of rectangles on a 2D plane, calculate the total area covered by them. Because rectangles can overlap, you cannot simply add their areas together.

The “Why” of the Difficulty: If you have two rectangles: * Rect A: (0,0) to (2,2) -> Area = 4 * Rect B: (1,1) to (3,3) -> Area = 4 If they overlap, the total area is not 8. You must subtract the intersection. With 200 rectangles, the intersections become a tangled mess of overlaps, triple-overlaps, and gaps.

2. Solution Explanation

The L5/L6 approach uses a **Sweep Line with Coordinate Compression**.

The Intuition: The Vertical Slicer Imagine a giant vertical line moving from left to right across the plane. Every time it hits the *start* or *end* of a rectangle, it stops. Between two stops, the “height” of the coverage doesn’t change.

1. **Vertical Slices:** We only care about x-coordinates where a rectangle begins or ends. These create vertical “strips.”
2. **Horizontal Coverage:** For each strip, we calculate how much of the vertical y-axis is covered by rectangles active in that strip.
3. **Area calculation:** Area = Width of Strip * Total Height covered in that strip.

Step-by-Step ASCII Visualization:

Input: `[[0,0,2,2], [1,1,3,3]]` (Format: `[x1, y1, x2, y2]`)

=====

1. IDENTIFY X-EVENTS (Where the sweep line stops)

=====

Rect 1: starts at x=0, ends at x=2

Rect 2: starts at x=1, ends at x=3

Sorted unique X-coordinates: `[0, 1, 2, 3]`

These define 3 vertical strips:
 Strip 1: x from 0 to 1 (Width = 1)
 Strip 2: x from 1 to 2 (Width = 1)
 Strip 3: x from 2 to 3 (Width = 1)

2. THE SWEEP (Moving Left to Right)

STRIP 1 (x: 0 to 1)

 Active Rectangles: Rect 1 (y: 0 to 2)
 Vertical segments: [[0, 2]]
 Total Height Covered: 2
 Area += (Width 1 * Height 2) = 2

```

      y
      ^
2 | [ ]
1 | [ ]
0 +-----> x
    0 1

```

STRIP 2 (x: 1 to 2)

 Active Rectangles:
 - Rect 1 (y: 0 to 2)
 - Rect 2 (y: 1 to 3)
 Vertical segments: [[0, 2], [1, 3]]
 Combined Height: 0 to 3 (Total Height = 3)
 Area += (Width 1 * Height 3) = 3

```

      y
      ^
3 |  [ ]
2 | [X][ ] (X = Overlap)
1 | [X][ ]
0 +-----> x
    1 2

```

STRIP 3 (x: 2 to 3)

 Active Rectangles: Rect 2 (y: 1 to 3)
 Vertical segments: [[1, 3]]
 Total Height Covered: 2
 Area += (Width 1 * Height 2) = 2

```

      y
      ^
3 |  [ ]
2 |  [ ]
1 |  [ ]
0 +-----> x
    2 3

```

3. FINAL CALCULATION

Total Area = 2 + 3 + 2 = 7

3. Time and Space Complexity Analysis

Let N be the number of rectangles.

TIME COMPLEXITY DERIVATION: $O(N^2)$

1. Collect and sort unique X-coordinates:

2N coordinates $\rightarrow O(N \log N)$

2. Outer Loop: Iterate through X-strips:

Loops 2N times. $\rightarrow O(N)$

3. Inner Process:

a. Find all rectangles active in current strip: $\rightarrow O(N)$

b. Sort and merge Y-intervals for active rects: $\rightarrow O(N \log N)$

Total Time = $O(N)$ strips * $O(N \log N)$ processing per strip

Total Time = $O(N^2 \log N)$

(Note: With a Segment Tree, an L6 could optimize this to $O(N \log N)$)

TIME COMPLEXITY DIAGRAM:

X-strips (2N) $\rightarrow$

Y-intervals per strip | x | x | x | x | x | (N log N)

Result: $O(N^2 \log N)$

SPACE COMPLEXITY DERIVATION: $O(N)$

What we store:

- List of X-events: 2N entries $\rightarrow O(N)$

- List of active Y-intervals: Max N entries ---> $O(N)$
- Unique X-coordinates: 2N entries ---> $O(N)$

Total Space = $O(N)$

4. Solution Code

Python Solution (Sweep Line + Interval Merging)

```
def rectangleArea(rectangles):
    # Modulo as required by the problem
    MOD = 10**9 + 7

    # Step 1: Collect all unique X coordinates
    x_coords = set()
    for x1, y1, x2, y2 in rectangles:
        x_coords.add(x1)
        x_coords.add(x2)
    sorted_x = sorted(list(x_coords))

    total_area = 0

    # Step 2: Iterate through the vertical strips created by sorted X coords
    for i in range(len(sorted_x) - 1):
        width = sorted_x[i+1] - sorted_x[i]
        if width == 0: continue

        # Step 3: Find all Y-intervals active in this X-strip
        active_y_intervals = []
        for x1, y1, x2, y2 in rectangles:
            # If the rectangle covers this entire strip width-wise
            if x1 <= sorted_x[i] and x2 >= sorted_x[i+1]:
                active_y_intervals.append((y1, y2))

        # Step 4: Calculate combined height of active Y-intervals
        # This is a mini 'Merge Intervals' problem
        active_y_intervals.sort()

        merged_height = 0
        current_y_start = -1
        current_y_end = -1

        for y_start, y_end in active_y_intervals:
            # Standard interval merging logic
            if y_start > current_y_end:
```

```

        merged_height += (current_y_end - current_y_start)
        current_y_start = y_start
        current_y_end = y_end
    else:
        current_y_end = max(current_y_end, y_end)

    # Add the last interval's height
    merged_height += (current_y_end - current_y_start)

    # Add strip's contribution to total area
    total_area += width * merged_height

return total_area % MOD

```

JavaScript Solution

```

/**
 * @param {number[][]} rectangles
 * @return {number}
 */
var rectangleArea = function(rectangles) {
    const MOD = 1000000007n; // Using BigInt for safety

    // Get unique X-coordinates
    let xCoords = new Set();
    for (const [x1, y1, x2, y2] of rectangles) {
        xCoords.add(x1);
        xCoords.add(x2);
    }
    let sortedX = Array.from(xCoords).sort((a, b) => a - b);

    let totalArea = 0n;

    for (let i = 0; i < sortedX.length - 1; i++) {
        let width = BigInt(sortedX[i + 1] - sortedX[i]);
        if (width === 0n) continue;

        // Find active Y-intervals for this strip
        let activeY = [];
        for (const [x1, y1, x2, y2] of rectangles) {
            if (x1 <= sortedX[i] && x2 >= sortedX[i + 1]) {
                activeY.push([y1, y2]);
            }
        }

        // Merge Y intervals

```



```

    activeY.sort((a, b) => a[0] - b[0]);
    let mergedHeight = 0n;
    let curStart = -1;
    let curEnd = -1;

    for (const [yStart, yEnd] of activeY) {
        if (yStart > curEnd) {
            mergedHeight += BigInt(Math.max(0, curEnd - curStart));
            curStart = yStart;
            curEnd = yEnd;
        } else {
            curEnd = Math.max(curEnd, yEnd);
        }
    }
    mergedHeight += BigInt(Math.max(0, curEnd - curStart));

    totalArea = (totalArea + width * mergedHeight) % MOD;
}

return Number(totalArea);
};

```

Note 1: Terminology and Techniques

- **Sweep Line:** A conceptual line that “sweeps” across the plane. It transforms a static 2D problem into a series of 1D problems.
 - **Coordinate Compression:** Instead of treating every point on the plane (which could be billions), we only treat the “interesting” points where something changes (the edges of rectangles).
 - **Interval Merging:** A technique to find the union of multiple overlapping ranges. It usually involves sorting by the start point and extending a “current end” pointer.
-

Note 2: Real-World / Interview Variations

1. **Sky Line Problem (Google / Bloomberg)** * **The Context:** Given buildings as [left, right, height], find the outline of the skyline. * **L5 Solution:** Use a Sweep Line and a Max-Heap. Sweep left to right. When a building starts, add its height to the heap. When it ends, remove it. The current highest value in the heap is the skyline.
2. **Ad Impression Overlap (Meta)** * **The Context:** An advertiser has multiple ad campaigns running. They want to know the total unique time their ads were visible on a user’s screen. * **L5 Solution:** This is exactly the 1D

version of Rectangle Area II. You have a list of time intervals `[start, end]`. You sort by start time and merge intervals to find the total union length.

3. Efficient Heatmap Generation (Google Maps) * **The Context:** If you have many overlapping “coverage circles” or “signal blocks” from Wi-Fi routers, how do you calculate the total square footage of coverage? * **L5 Solution:** If the blocks are rectangles, you use the Sweep Line + Segment Tree approach for high efficiency. If they are circles, you might approximate them with many small rectangles or use more complex calculus-based geometry.

Would you like to explore the $O(N \log N)$ **Segment Tree** optimization for this problem? It’s often the follow-up question for L6 candidates.

1854. Maximum Population Year

Solving this problem at a Google L5/L6 level involves moving beyond a “brute force” nested loop and identifying it as a classic **Line Sweep** or **Difference Array** problem.

Senior engineers recognize that we don’t care about every single day; we only care about the **events** that change the population: births and deaths.

1. Problem Explanation

The Goal: You are given a list of logs where each log is `[birth, death]`. A person is considered alive in every year from `birth` to `death - 1`. You need to find the earliest year with the maximum population.

The “Why” of the Logic: If someone is born in 1950 and dies in 1961, they contribute +1 to the population for every year from 1950 to 1960. They do **not** count in 1961.

The Brute Force (Junior) Approach: Create a map of every year and for every person, loop through every year of their life and increment a counter. * *Problem:* If there are 100 people living 100 years, you do 10,000 operations. It’s inefficient.

The L5/L6 (Optimized) Approach: We only mark the **changes**. * A birth is a +1 event. * A death is a -1 event. * We then walk through time once, adding up these changes (Calculating the “Prefix Sum”).

2. Solution Explanation

We use a “Difference Array” of size 101 (since the problem constraints say years are between 1950 and 2050).

Step-by-Step Visualization Input: [[1950, 1960], [1950, 1970],
[1960, 1970]]

=====

INITIAL STATE (Difference Array)

=====

Index 0 represents 1950, Index 10 represents 1960, etc.

Array (all zeros): [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...]

=====

STEP 1: Process Log [1950, 1960]

=====

- Birth (1950): Index 1950 - 1950 = 0. Add +1.

- Death (1960): Index 1960 - 1950 = 10. Add -1.

Array: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, -1, ...]

=====

STEP 2: Process Log [1950, 1970]

=====

- Birth (1950): Index 0. Add +1 (Total = 2).

- Death (1970): Index 20. Add -1.

Array: [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, -1, ..., -1, ...]

=====

STEP 3: Process Log [1960, 1970]

=====

- Birth (1960): Index 10. Add +1 (Total = 0).

- Death (1970): Index 20. Add -1 (Total = -2).

Final Difference Array:

Index: 0 1 ... 10 ... 20

Value: [2, 0, ..., 0, ..., -2]

=====

STEP 4: Calculate Running Population (Prefix Sum)

=====

We walk through the array and keep a running total.

Year 1950 (Idx 0): Total = 2. Max Pop = 2 (Year 1950)

Year 1951-1959: Total stays 2. Max Pop = 2

Year 1960 (Idx 10): Total = 2 + 0 = 2. Max Pop = 2

Year 1961-1969: Total stays 2. Max Pop = 2

Year 1970 (Idx 20): Total = 2 + (-2) = 0. Max Pop = 2

Earliest Year with Max Pop (2) is 1950.

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(N + Y)$

Let N = Number of logs (people).

Let Y = Number of years (range size, here 100).

1. Process Logs:

Loop through N logs once.

Each update is $O(1)$. ---> $O(N)$

2. Calculate Max:

Loop through Y years once.

Each calculation is $O(1)$. ---> $O(Y)$

Total Time = $O(N + Y)$

Since Y is constant (100), this is essentially $O(N)$.

SPACE COMPLEXITY DERIVATION: $O(Y)$

We store a fixed-size array to track the years.

The size is exactly 101.

Calculation:

Space = 101 integers.

Total Space = $O(Y)$

Since Y is constant, this is essentially $O(1)$.

4. Solution Code

Python Implementation

```
def maximumPopulation(logs):
    # We only need to track years from 1950 to 2050 (101 years)
    # This is our Difference Array
    year_delta = [0] * 101

    # Fill the difference array
    for birth, death in logs:
        # Increment birth year
```

```

    year_delta[birth - 1950] += 1
    # Decrement death year (person is NOT alive in this year)
    year_delta[death - 1950] -= 1

max_pop = 0
max_year = 1950
current_pop = 0

# Calculate prefix sums to find the max population
for i in range(len(year_delta)):
    current_pop += year_delta[i]

    # We want the EARLIEST year, so we only update if
    # current_pop is strictly GREATER than max_pop.
    if current_pop > max_pop:
        max_pop = current_pop
        max_year = 1950 + i

return max_year

```

JavaScript Implementation

```

/**
 * @param {number[][]} logs
 * @return {number}
 */
var maximumPopulation = function(logs) {
    // Range is 1950 to 2050
    const yearDelta = new Array(101).fill(0);

    // Step 1: Mark Birth and Death events
    for (let [birth, death] of logs) {
        yearDelta[birth - 1950]++;
        yearDelta[death - 1950]--;
    }

    let maxPop = 0;
    let maxYear = 1950;
    let currentPop = 0;

    // Step 2: Sweep through the timeline
    for (let i = 0; i < yearDelta.length; i++) {
        currentPop += yearDelta[i];

        // If we find a higher population, record the year
        if (currentPop > maxPop) {

```

```

        maxPop = currentPop;
        maxYear = 1950 + i;
    }
}

return maxYear;
};

```

Note 1: Terminology and Techniques

- **Difference Array / Line Sweep:** Instead of updating a whole range of values, you update the **boundaries**. By marking the start and end of an interval, you can reconstruct the state of any point in time by summing all events from the beginning.
 - **Prefix Sum:** The cumulative sum of an array. In this problem, the prefix sum of “changes” gives us the “actual value” at any point.
-

Note 2: Real World / Interview Variations

1. Variation: Meeting Rooms II (Google/Bloomberg) * The Twist: Given meeting times `[[start, end]]`, find the minimum number of rooms needed. * *How to solve:* This is the exact same problem! A meeting starting is `+1` room, and a meeting ending is `-1` room. The maximum value in your prefix sum is the number of rooms needed.

2. Variation: Corporate Flight Bookings (Meta) * The Twist: You have bookings for ranges of seats on a flight `[first, last, seats]`. Calculate total seats for each flight. * *How to solve:* Use the Difference Array. For a booking `[i, j, k]`, add `k` to index `i` and subtract `k` from index `j+1`. Then calculate the prefix sum.

3. Variation: Skyline Problem (Hard - Google) * The Twist: Given building positions and heights, find the silhouette (skyline). * *How to solve:* This is a more complex Line Sweep. You sort events (left and right edges of buildings). As you sweep across the x-axis, you use a Max-Heap to keep track of the current tallest building. When the tallest building changes, you record a new point in your skyline.